

보행 보조용 분할 모델 실험: 상세 보고서

인수인계 문서 전체 + 부록 (오류 분석, 속도 측정, 데이터 예시)

Team Insight · detection 브랜치 · 2026-10-04 · 요약본: 보행보조_분할모델_실험보고서.pdf

이 문서만 읽고 작업을 이어받을 수 있도록 쓴 보고서다. 무엇을 했고, 무엇이 되고 안 되는지, 왜 그런지, 숫자는 어디서 나왔는지, 어떻게 다시 돌리는지를 모두 담았다. 숫자 옆의 경로는 그 숫자를 만든 파일이다. 같은 명령을 다시 실행하면 같은 숫자가 나온다.

작성: 2026-10-04. 모든 실험·분석 완료.

0. 한 페이지 요약

질문	답
무엇을 하나	시각장애인 보행 보조용 분할 모델(YOLO11s-seg)에 Foundation Model(FM) 인코더를 붙이면 성능이 오르는지 실험
데이터	AI Hub 「인도보행 영상」(dataSetSn 189). 라벨된 사진 92,772장, 영상 1,955개, 객체 581,529개
1차 결과 (10클래스, 소량 데이터)	FM을 추론에도 쓰는 fusion은 7개 모두 baseline보다 좋음 (mask mAP50-95 0.283 → 0.313~0.358). FM을 학습에만 쓰는 distill은 효과 없음 (±0.008)
최종 결과 (10클래스, test 13,721장)	mask mAP50-95: E0 0.308 → A5(C-RADIOv3 fusion) 0.331 (+0.023) . 10개 중 9개 클래스에서 A5가 높음
추론 속도 (A5000, 640, batch 1)	E0 13.4ms(75 FPS) vs A5 22.5ms(44 FPS) , 실제 이미지 end-to-end 기준. CPU 4스레드에서는 E0 0.29초 vs A5 2.87초 (약 10배)
mAP가 낮은 이유 (실제 이미지와 오류 분석으로 확인)	① 라벨 누락 : 모델이 찾은 라바콘·기둥·벤치·신호등 뒷면·Polygon 영상의 계단이 정답에 없어 오검 출로 처리됨 → 실제 성능은 숫자보다 좋음 ② 작은 객체 : 짧은 변 16px 미만은 대부분 못 찾음, traffic_light는 89%가 16px 미만 ③ scooter는 데이터 부족 + 오토바이·자전거와 혼동 (train 144장, AP 0.04) ④ stairs 라벨 오류 (원본의 상당수가 맨홀·보호판·연석)
문제가 아닌 것	흐림(흐린 객체 0.2%, 대부분 야간), 어두운 프레임(0.4%)
가장 먼저 할 일	배포 기기(노트북 CPU)에서 속도 실측 → fusion 사용 여부 결정 → stairs 재라벨링, scooter 데이터 보강

1. 프로젝트와 이 브랜치의 역할

- 레포: github.com/sangjune2001/A-Proactive-Egocentric-Streaming-Video-Assistant-for-Blind-Navigation
- 전체 시스템: 1인칭 스트리밍 영상 → 경량 모듈(YOLO-seg + Trigger)이 매 프레임 감시 → 위험할 때만 VLM 호출 → TTS로 안내.
- 이 브랜치(detection)**는 그중 **YOLO-seg 부분**: 데이터셋 구축(재라벨링), YOLO11s-seg 학습, FM 인코더 비교. `main`에는 아직 병합하지 않았다.
- 실시간 보조가 목적이므로 **성능과 추론 속도를 같이** 봐야 한다. 최종 배포는 노트북(그랩) CPU + OpenVINO를 가정했다(`insight_fm/export_plain.py` 주석).

2. 데이터

2.1 출처와 위치

무엇	어디	비고
원본 영상 프레임 (Polygon)	gdrive:sideguide/polygon/P1.zip ~ P14.zip	zip당 약 10GB, 합계 약 134GB. 1920×1080
원본 노면 데이터 (Surface)	gdrive:sideguide/surface/S1.zip	stairs 라벨은 여기서만 나옴
학습용 라벨	gdrive:aihub189_yolo/labels_all.jsonl (243MB)	29종 원본 라벨을 10클래스로 재라벨링한 YOLO-seg 폴리곤. Colab에서 만들
원본 라벨 개수표	gdrive:sideguide/class_counts.csv	AI Hub 원본 라벨별 객체·사진·폴더 수
라벨을 만든 기존 모델	gdrive:aihub189_yolo/runs/seg10/	클래스 순서의 정답(7=obstacle, 8=stairs)은 이 모델의 names 로 확인

labels_all.jsonl 한 줄 형식:

```
{ "k": "Polygon_0001__MP_SEL_P000002",           // 고유 키 = <영상 폴더>__<파일명>
  "z": "/content/drive/MyDrive/sideguide/polygon/P1.zip", // 원본 zip (MyDrive/ 뒤가 Drive 경로)
  "m": "Polygon_0001/MP_SEL_P000002.jpg",         // zip 안 경로
  "l": ["7 0.28 0.00 0.29 0.34 ...", ...]         // YOLO-seg: 클래스 x1 y1 x2 y2 ... (0~1 정규화)
```

2.2 클래스 (10개)와 원본 매핑

0 person, 1 bicycle, 2 scooter, 3 motorcycle, 4 car, 5 bus, 6 other_vehicle, 7 obstacle, 8 stairs, 9 traffic_light

- **obstacle** = 원본 15종 합침: pole 98,463 / tree_trunk 97,018 / bollard 37,266 / movable_signage 20,203 / potted_plant 8,031 / bench 6,215 / power_controller 5,262 / barricade 4,323 / stop 4,311 / traffic_light_controller 3,986 / chair 3,643 / fire_hydrant 2,593 / table 1,317 / kiosk 1,198 / parking_meter 92 → 합계 293,921 (학습 데이터 293,918과 일치)
- **other_vehicle** = truck 33,211 + carrier 1,479 + stroller 485 + wheelchair 204
- **stairs** = Surface의 caution_zone[stairs] 399개 (Polygon에는 계단 클래스가 없음)
- **안 쓰는 원본 라벨**: traffic_sign 38,918개(꽤 많음), dog 166, cat 51
- 전체 매핑표: [insight_fm/analysis/data_analysis.md](https://insight-fm.github.io/analysis/data_analysis.md) 7절

2.3 규모와 분할

	영상	사진	객체
전체	1,955	92,772	581,529 (사진당 6.3개)
train	1,368	64,952	403,570
val	293	14,099	90,024
test	294	13,721	87,935

- 영상 단위 70/15/15 분할: 같은 영상의 프레임이 train과 test에 섞이지 않는다(섞이면 성능이 부풀려짐).
- 50개 seed 중 val-test에 드문 클래스가 가장 많이 들어가는 seed(34)를 자동 선택. pilot-최종 학습 모두 같은 분할을 쓴다.

2.4 클래스별 개수: 객체 수 ≠ 사진 수

"객체 수"는 폴리곤 개수, "사진 수"는 그 클래스가 하나라도 있는 사진 수다. 신호등 3개가 있는 사진은 객체 3, 사진 1.

클래스	객체	사진	영상	사진당 객체	train 객체 / 사진	test 객체 / 사진
obstacle	293,918	82,745	1,840	3.55	203,896 / 57,977	43,540 / 12,123
car	147,131	55,957	1,800	2.63	102,448 / 39,117	22,566 / 8,307
person	47,192	26,149	1,691	1.80	32,735 / 18,008	7,174 / 4,054
other_vehicle	35,379	21,516	1,681	1.64	24,645 / 14,984	5,317 / 3,286
traffic_light	26,796	11,106	1,374	2.41	18,598 / 7,750	4,193 / 1,680
bus	11,321	6,484	1,323	1.75	7,526 / 4,331	1,978 / 1,138
bicycle	10,003	6,581	1,367	1.52	6,969 / 4,566	1,600 / 1,031
motorcycle	9,039	6,731	1,261	1.34	6,292 / 4,692	1,394 / 1,025
stairs	399	372	115	1.07	264 / 247	79 / 72
scooter	351	224	124	1.57	197 / 144	94 / 33

- AI Hub 원본 표(scooter 351개 / 224장 / 124폴더)와 정확히 일치한다. **scooter는 train에 144장뿐**이고 test에는 33장이라 test AP 변동이 크다.
- 가장 많은 obstacle과 가장 적은 scooter의 객체 수는 약 837배 차이.
- 출처: `insight_fm/analysis/class_stats_all.csv`, `class_stats_by_split.csv` (python analyze_data.py)

2.5 카메라와 촬영 높이

카메라	사진	비율	추정 높이
스마트폰 (MP_SEL, MP_KSC, MP_TW)	53,003	57%	가슴 높이 (약 1.3~1.5m, 손에 들고 촬영)
ZED 스테레오 (ZED1 ~ ZED4)	39,397	43%	허리~가슴 아래 (약 0.8~0.9m, 거치 촬영 추정)
Surface 스마트폰	372	0.4%	바닥을 향해 촬영

- 높이는 공식 수치가 아니라 **추정치**다. 사진 속 보행자(약 1.7m)와 지평선 위치를 비교해 계산했다(사진 2장 기준, 경사로에서는 오차 큼). AI Hub 소개 페이지에는 촬영 높이가 없고, 데이터 설명서 PDF에 있을 수 있다.
- 두 카메라 모두 약간 아래를 향해 화면 아래쪽 절반 가까이가 보도다. 예시: `insight_fm/analysis/camera_samples.jpg`
- **배포 시 카메라를 가슴 높이에 다는 것이 학습 데이터와 가장 잘 맞는다.** 머리-안경 높이에서는 시점이 달라 성능이 떨어질 수 있다.

3. 실제 이미지로 본 객체 상태

학습에 쓰는 640px 이미지에서 객체 58만 개를 하나씩 측정했다(`insight_fm/analyze_objects.py`, 결과 [analysis/objects/object_quality.md](#), 원본 측정값 `gdrive:sideguide/analysis/objects.csv.gz`).

클래스	박스 짧은 변 중앙값	아주 작음 (<16px)	잘림 (가장자리에 닿음)	흐림	어두움 (평균 밝기<50)
traffic_light	8px	88.8%	6.8%	0.1%	26.9%
obstacle	16px	47.9%	30.3%	0.6%	21.6%
person	18px	39.0%	5.9%	0.2%	24.2%
scooter	25px	30.8%	7.1%	0%	17.4%
other_vehicle	34px	16.6%	17.4%	0.1%	15.3%
stairs	70px	15.8%	65.4%	0%	0%
car	30px	18.9%	17.0%	0%	13.4%

해석(예시 이미지를 직접 보고 확인함):

1. **작은 객체가 가장 큰 문제.** traffic_light는 절반이 짧은 변 8px 이하다(예: 14×4px, 7×3px). 640px 입력에서는 신호등 색도 구분하기 어렵다. obstacle도 기둥처럼 **폭 5~20px인 가늘고 긴 객체**가 많아 마스크를 정확히 맞추기 어렵다.
2. **잘림은 대부분 자연스러운 경우.** 기둥·나무처럼 키 큰 객체가 화면 위로 잘린 것이고(카메라가 낮아서), 보이는 부분은 제대로 라벨되어 있다. stairs의 65%가 잘린 것도 계단이 화면 가장자리에 크게 걸치기 때문이다. 성능 저하 요인으로선 작다.
3. **흐림은 문제가 아님.** 흐린 객체 926개(0.2%)는 대부분 야간·역광의 어두운 장면이다. 움직임 블러는 거의 없다. (흐림 지표는 객체 박스 안 Laplacian 분산 < 50. JPEG 노이즈 때문에 절대값보다는 "거의 없다"는 결론만 믿을 것.)
4. **"어두움"은 조명보다 물체 색.** 어두운 객체의 대부분은 검은 기둥·나무 줄기·그늘 속 물체다. 어두운 프레임 자체는 0.4%뿐이다.
5. 클래스별 실제 모습: `analysis/objects/sheet_<클래스>.jpg`, 문제 유형별: `problem_<tiny|truncated|blurry|dark>.jpg`

4. ⚠ 라벨 품질 문제 (원본 AI Hub)

4.1 stairs 라벨의 상당수는 계단이 아니다

- 학습 데이터의 stairs 399개는 **원본 XML의** `caution_zone[stairs]` **399개와 좌표까지 전부 일치한다**(변환 코드는 정확함).
- 그런데 원본 이미지에 원본 폴리곤을 그려 보면 **실제 계단은 3분의 1 정도**이고, 나머지는 **가로수 보호판, 맨홀, 연석·턱, 차도 경계석**에 stairs가 붙어 있다. 즉 **AI Hub 원본 작업자의 라벨 오류**다.
- 증거 이미지: [insight_fm/analysis/stairs_label_noise.jpg](#) (빨간 선 = 원본 stairs 폴리곤, 무작위 12장)
- 게다가 **Polygon 영상(전체의 99.6%)에는 계단 클래스 자체가 없어서**, 그 영상에 찍힌 계단은 배경으로 학습된다.
- 결론: **stairs는 데이터를 늘려도 좋아지지 않는다. 재라벨링이 먼저다.** 372장이자 직접 고치는 데 몇 시간이면 된다.
- 확인 스크립트: `insight_fm/tools/s1chk.py` (원본 XML 라벨 집계), `s1match.py` (학습 라벨 ↔ 원본 폴리곤 대조), `s1draw.py` (원본 폴리곤 그리기). S1.zip을 `~/sg/s1chk/` 에 받아 두고 실행

4.2 기타 라벨 특성

- **obstacle = 15종 혼합**: 기둥·나무·볼라드·입간판·화분·벤치·키오스크... 생김새가 전혀 달라 하나의 클래스로 배우기 어렵다. 보행 보조 관점에서는 "부딪힐 수 있는 것"이라 합친 것으로 보이며, 성능을 올리려면 2~3개 그룹(기둥형 / 낮은 장애물 / 큰 구조물)으로 나누는 것을 검토할 만하다.
- **other_vehicle**: 대부분 트럭. 일부는 승합차·SUV처럼 car와 경계가 애매하다.
- **scooter**: 공유 전동킥보드, 어린이 킥보드, 전동 휠까지 포함. 개수가 너무 적다.
- **traffic_light**: 보행자·차량 신호등, 앞면·뒷면(불 꺼진 뒷면 포함)이 섞여 있다.

5. 실험

5.1 방식과 ID

- **A = Fusion**: FM feature를 YOLO 백본 P3/P4/P5에 더함(마지막 conv 0 초기화 → 시작 시점은 원래 YOLO와 동일). **추론할 때도 FM이 돈다** → 느림.
- **B = Distill**: FM은 학습할 때 teacher로만 쓰고, YOLO feature가 FM feature를 닮도록 cosine 손실 추가. **추론은 순수 YOLO11s-seg** → baseline과 같은 속도. `export_plain.py` 로 보조 head를 떼어냄.
- **E0** = FM 없는 baseline. 번호는 인코더: 1 SigLIP2-B, 2a DINOv2-B, 2b DINOv3-B, 3a/3b 두 인코더 결합, 4 RADIOv2.5-B(비상업), 5 C-RADIOv3-B(상업 가능), 6 C-RADIOv4-SO400M(4억 파라미터).
- 모든 FM은 frozen. 공통: YOLO11s-seg COCO 사전학습 가중치, imgsz 640, copy_paste 0.3, AMP, seed 0.

5.2 1차 pilot (10클래스, 2026-10-01, A5000)

설정: train 996장(클래스마다 그 클래스가 든 **사진** 약 200장) / val 443장, 30 epoch, pilot val로 평가.

ID	방식	인코더	mask mAP50-95	E0 대비	지연 (ms, A5000 FP16 bs1)
A6	fusion	C-RADIOv4-SO400M	0.358	+0.074	47.8
A2a	fusion	DINOv2-B	0.336	+0.052	22.3
A2b	fusion	DINOv3-B	0.332	+0.049	27.1
A3a	fusion	DINOv2-B + SigLIP2-B	0.332	+0.048	34.0
A5	fusion	C-RADIOv3-B	0.327	+0.043	20.8
A3b	fusion	DINOv3-B + SigLIP2-B	0.326	+0.042	46.9
A4	fusion	RADIOv2.5-B	0.318	+0.035	20.8
A1	fusion	SigLIP2-B	0.313	+0.030	28.6
B5	distill	C-RADIOv3-B	0.286	+0.003	13.1
B4	distill	RADIOv2.5-B	0.285	+0.001	10.7
E0	baseline	–	0.283	0	10.4
B3a / B2a / B6 / B1 / B3b / B2b	distill	(각각)	0.276 ~ 0.281	-0.003 ~ -0.008	10.3 ~ 12.9

전체 표·클래스별 AP: [insight_fm/README.md](#), 원본: `insight_fm/results/pilot_summary.csv`, 모델·로그: `gdrive:sideguide/runs/<ID>_s0/`.

되는 것

- fusion은 7개 모두 확실히 오른다(+0.03 이상, seed 노이즈보다 큼). 많이 오른 클래스: other_vehicle(0.24 → 0.33~0.40), stairs, motorcycle, bus.
- 실용 후보: A2a(DINOv2), A5(C-RADIOv3). 둘 다 지연 약 2배. 둘의 차이 0.009는 노이즈 범위.

안 되는 것

- distill은 8개 모두 효과 없음(teacher를 SO400M으로 키운 B6도 마찬가지). 학습 데이터 996장·30 epoch· $\lambda=1$ 이라는 조건에서는 FM 지식이 전달되지 않았다. 최종 학습에서는 제외했다(데이터·epoch를 늘렸을 때 달라지는지는 확인하지 않음).
- 인코더 두 개 결합(A3a/A3b)은 하나보다 낫지 않고 느리다.
- scooter-traffic_light는 모든 모델에서 AP 0.1~0.19.

한계: seed 1개, val 443장 → ± 0.01 이하 차이는 믿지 말 것. 지연은 GPU 기준이고, 노트북 CPU에서는 fusion이 훨씬 더 느려진다(6장).

5.3 최종 학습 (2026-10-04)

항목	설정
클래스	10개 모두 유지 (person, bicycle, scooter, motorcycle, car, bus, other_vehicle, obstacle, stairs, traffic_light). 관심 클래스는 scooter-stairs-obstacle-other_vehicle-traffic_light
train	scooter-stairs-traffic_light가 든 train 사진 전부 = 8,120장. 다른 클래스도 이 사진들에 이미 200장 이상씩 들어 있음
val	클래스당 사진 약 100장 (best epoch 선택용)
test	전체 test 13,721장 (학습에 한 번도 안 쓴 데이터로 최종 점수)
학습	E0: 100 epoch, patience 30 / A5(C-RADIOv3 fusion): 70 epoch, patience 15. imgsz 640, seed 0. 두 모델을 같은 GPU에서 동시에 학습(학습 시간은 비교 대상 아님). distill은 1차 pilot에서 효과가 없어 제외
명령	insight_fm/tools/run_final10.sh (E0-A5 동시 학습 → 오류 분석 → 추론 속도 측정)
결과 위치	서버 <code>~/repo/insight_fm/runs10/final/</code> , Drive <code>gdrive:sideguide/runs_final10/</code>

ID	mask mAP50-95 (test)	mAP50	scooter	stairs	obstacle	other_vehicle	traffic_light
E0	0.308	0.534	0.037	0.219	0.363	0.379	0.165
A5	0.331	0.568	0.040	0.210	0.376	0.414	0.176

10개 클래스 전체 (mask AP50-95):

클래스	E0	A5	차이
car	0.577	0.586	+0.009
person	0.437	0.463	+0.026
bus	0.390	0.428	+0.038
other_vehicle	0.379	0.414	+0.035
obstacle	0.363	0.376	+0.013
motorcycle	0.302	0.378	+0.075
bicycle	0.209	0.236	+0.027
stairs	0.219	0.210	-0.009
traffic_light	0.165	0.176	+0.011
scooter	0.037	0.040	+0.003
전체	0.308	0.331	+0.023

- E0: 100 epoch 완주, best epoch 73. A5: 61 epoch에서 patience로 정지, best epoch 56. 둘 다 best 가중치로 평가.
- 학습 곡선: [analysis/final/learning_curve.png](#). E0는 73 epoch 이후 val loss가 다시 오른다(과적합). A5는 처음부터 E0보다 val 점수가 높고 더 빨리 수렴한다.
- 혼동 행렬: [analysis/final/confusion_E0.png](#), [confusion_A5.png](#). 원본 수치: [analysis/final/final_summary.csv](#), [done_E0.json](#), [done_A5.json](#)
- 모델 가중치: [gdrive:sideguide/runs_final10/<E0_s0|A5_s0>/weights/best.pt](#)
- 1차 pilot(train 996장, 30 epoch, pilot val)보다 점수가 높은 것은 학습 데이터 8배·epoch 증가 효과이며, 평가셋이 달라 직접 비교는 하지 않는다.

5.4 오류 분석: mAP가 왜 낮은가 (최종 E0 기준, A5도 경향 동일)

test 전체에서 정답 객체마다 찾았는지(confidence ≥ 0.25 , 같은 클래스 박스 IoU ≥ 0.5), 예측마다 정답이 있었는지 판정했다([insight_fm/eval_errors.py](#)). 결과: [analysis/final/errors_E0/errors.md](#), [errors_A5/errors.md](#), 객체별 원본 [gdrive:sideguide/runs_final10/analysis/](#).

클래스	정답 객체	찾음 (recall)	오검출 (FP)	정답 1개당 FP	precision
person	7,174	74.9%	1,537	0.21	77.8%
car	22,566	83.3%	6,815	0.30	73.4%
obstacle	43,540	65.0%	14,844	0.34	65.6%
other_vehicle	5,317	57.8%	1,682	0.32	64.6%
bus	1,978	55.0%	528	0.27	67.3%
motorcycle	1,394	66.3%	799	0.57	53.6%
bicycle	1,600	57.6%	680	0.42	57.6%
traffic_light	4,193	78.6%	3,787	0.90	46.5%
stairs	79	43.0%	80	1.01	29.8%
scooter	94	36.2%	312	3.32	9.8%

원인 1. 라벨 누락 → 맞게 찾아도 오검출로 처리 (평가 수치를 깎음)

- **obstacle** 오검출 상위 40개는 대부분 **실제 장애물**(라바콘, 고가 기둥, 벤치, 분전함, 입간판, 볼라드)인데 정답 라벨이 없다(errors_E0/fp_obstacle.jpg). 라바콘은 AI Hub 원본 라벨 목록에 아예 없다.
- **traffic_light** 오검출은 대부분 **라벨 안 된 신호등**(옆면·뒷면·멀리 있는 것)과 **횡단보도 표지판**이다(fp_traffic_light.jpg). traffic_sign을 학습에서 뺐기 때문에 표지판을 구분할 근거가 없다.
- **stairs** 오검출에는 **진짜 계단이 많다**(Polygon 영상에는 계단 라벨이 없음). 나머지는 연석·맨홀·배수구로, 학습 라벨(4.1절)이 그렇게 가르쳤다(fp_stairs.jpg).
- → 모델보다 **평가 데이터의 한계**다. 실제 성능은 숫자보다 좋다. 정확히 재려면 test 일부라도 라벨을 보완해야 한다.

원인 2. 작은 객체 (크기별 recall, 박스 짧은 변 기준)

클래스	0~8px	8~16px	16~32px	32~64px	64px 이상
person	25.8%	68.4%	83.0%	88.6%	94.2%
car	16.3%	65.6%	87.2%	93.2%	97.0%
bicycle	0.0%	24.1%	56.1%	78.1%	82.0%
other_vehicle	3.2%	18.3%	53.4%	70.3%	81.9%
obstacle	41.4%	64.4%	73.2%	75.6%	74.8%
scooter	0.0%	10.5%	30.8%	70.4%	45.5%
traffic_light	75.4%	82.4%	89.1%	97.9%	—

- 16px 미만이면 대부분 클래스가 크게 떨어진다. 데이터에서 16px 미만 비율은 traffic_light 89%, obstacle 48%, person 39%, scooter 31%(3장).
- **traffic_light는 8px 이하도 75%를 찾는데 mask AP50-95는 0.17**이다. 4~8px 객체는 마스크 경계가 1~2px만 어긋나도 IoU가 크게 떨어지기 때문. → 입력 해상도(960~1280)를 올리면 직접적으로 좋아질 부분.

원인 3. scooter: 데이터 부족 + 비슷한 클래스와 혼동

- train에 사진 144장(객체 197개)뿐이다. motorcycle 476장, bicycle 593장과 비교해 너무 적다.
- 오검출 312개 중 **49%가 다른 클래스 정답(오토바이·자전거·사람)과 겹친다**. 오토바이·자전거를 scooter로 잘못 부른다. 나머지는 손수레·유모차·어린이 탈것(fp_scooter.jpg).
- 놓친 scooter는 줄지어 세워진 공유 킥보드(가늘고 겹침)와 아주 작은 것이 대부분이다(missed_scooter.jpg). 3×1px 같은 잘못된 라벨도 섞여 있다.
- test 정답이 94개(사진 33장)뿐이라 AP 자체도 몇 개 차이로 크게 흔들린다.
- → 모델 구조로는 안 풀린다(A5도 0.040). **scooter 데이터 추가가 필요하다**.

원인 4. 과적합 (E0): 73 epoch 이후 val loss 상승. best 가중치를 쓰므로 결과에는 영향 없음.

원인이 아닌 것: 잘림(잘린 객체 recall이 오히려 높음, 큰 객체가 잘리기 때문), 밝기(어두운 객체가 조금 낮지만 other_vehicle 43% vs 61% 정도), 카메라 종류(대체로 스마트폰이 ZED보다 높지만 차이는 -2~+14%p, 큰 차이는 other_vehicle-bus).

6. 추론 속도 (E0 vs C-RADIO fusion A5)

학습이 모두 끝난 뒤 GPU가 빈 상태에서 모델을 하나씩 측정(insight_fm/bench_speed.py). 결과: [analysis/final/speed.md](#), speed.csv.

항목	E0 (YOLO11s-seg)	A5 (+ C-RADIOv3-B)	배수
파라미터 (YOLO + FM)	10.1M	108.3M	10.7x
연산량 (640×640 1장)	32.9 GFLOPs	408 GFLOPs	12.4x
GPU FP16, 모델만	11.1 ms	21.1 ms	1.91x
GPU FP32, 모델만	10.4 ms	20.2 ms	1.94x
GPU FP16, 8장 묶음 처리량	555 장/초	93 장/초	0.17x
실제 이미지 end-to-end (전처리+추론+NMS-마스크)	13.4 ms (75 FPS)	22.5 ms (44 FPS)	1.68x
CPU 4스레드, 모델만	294 ms (3.4 FPS)	2,867 ms (0.35 FPS)	9.8x
CPU 8스레드, 모델만	195 ms (5.1 FPS)	1,615 ms (0.6 FPS)	8.3x

- A5의 GPU 시간 21.1ms 중 **C-RADIO 인코더가 10.1ms(48%)**. CPU 4스레드에서는 C-RADIO만 2,539ms로 대부분을 차지한다.
- **GPU에서는 A5도 44 FPS로 실시간 가능**. 정확도 +0.023을 위해 속도를 약 절반으로 쓰는 셈.
- **CPU에서는 A5가 1초에 1장도 처리하지 못한다**. 노트북 CPU 배포가 목표라면 fusion은 그대로 쓸 수 없다 (E0도 CPU 4스레드 PyTorch로는 3.4 FPS라 OpenVINO 변환이 필요).
- CPU 수치는 서버 CPU의 스레드 수를 제한해 켜 값이다. 실제 그램 노트북과 다를 수 있어 **실측이 필요하다**. GPU 측정은 batch 1, 640×640, 210장 중 앞 10장을 워밍업으로 제외한 중앙값.

7. 재현 방법 (서버)

7.1 서버

- **Runyour AI(몬드리안)** GPU 대여. 접속: `ssh -i <발급받은 .pem> -p 22 ubuntu@machine.runyour.ai`. 인스턴스마다 .pem이 새로 나온다.
- 과금: **빌린 동안 분 단위(A5000 약 900원/시간)**. GPU를 안 써도, SSH를 끊어도 나간다. **반환해야 멈추고, 반환하면 서버의 데이터는 모두 사라진다**(같은 머신을 다시 빌려도 복구 안 됨). 일시정지 기능은 없다. 크레딧이 떨어지면 자동 반환된다.
- 크레딧: 학교에서 20만 원 충전. 1차 인스턴스 약 15시간(약 1.4만 원) 사용 후 반환. 2차 인스턴스(2026-10-03 21:56~) 사용 중.
- **서버 이미지가 매번 다를 수 있다**. 1차는 conda(Python 3.12) 있음, 2차는 순수 Ubuntu 22.04(Python 3.10, pip 없음) → `python3 -m venv ~/venv` 후 설치. 둘 다 Docker 컨테이너이고 `sudo` 비밀번호 없음.
- GPU 선택 참고: L20(48GB) > A10×2 ≈ A5000(24GB) > V100(CUDA 13 미지원, 비추천).

7.2 처음부터 돌리기

```
# 이 PC(WSL)에서: rclone 설정을 서버로 (Drive 접근용, 한 번만)
ssh -i a5000.pem ubuntu@machine.runyour.ai "mkdir -p ~/.config/rclone"
scp -i a5000.pem ~/.config/rclone/rclone.conf ubuntu@machine.runyour.ai:~/.config/rclone/

# 서버에서: 반드시 tmux 안에서 (오래 걸리는 SSH 명령은 서버가 끊는다)
git clone -b detection https://github.com/sangjune2001/A-Proactive-Egocentric-Streaming-Video-Assistant-for-Blind-Navigation.git ~/repo
tmux new -s exp
cd ~/repo/insight_fm
read -s -p "HF token: " HF_TOKEN && export HF_TOKEN; echo      # DIN0v3만 필요. history에 안 남음
bash setup.sh          # deps → data(약 1시간: zip 15개 받고 640px 추출, zip은 지움) → hf → prefetch →
selftest → build
python run_all.py --phase pilot          # 1차 pilot 재현
# 최종 학습 (10클래스, 8,120장, 100 epoch, test 평가): tools/run_final10.sh와 같음
python run_all.py --phase final --final-epochs 100 --final-patience 30 --ids E0
python run_all.py --phase final --final-epochs 70 --final-patience 15 --ids A5
python eval_errors.py --weights runs/final/E0_s0/weights/best.pt --data ~/sg/yolo/full --out analysis/errors_E0
python bench_speed.py --runs runs/final --images ~/sg/yolo/full/images/test --out results/speed
```

- `RCLONE_REMOTE_RUNS=gdrive:...` 를 export하면 epoch마다 결과가 Drive로 백업된다(반환 대비 필수).
- 끊겨도 같은 명령을 다시 치면 끝난 실험은 건너뛰고, 중단된 실험은 `last.pt` 에서 이어서 학습한다.
- `touch results/HOLD_<phase>` 로 해당 단계 직전에 멈출 수 있다.
- 데이터 분석 재생성: `python analyze_data.py --jsonl ~/sg/labels_all.jsonl --out analysis, python analyze_objects.py --jsonl ~/sg/labels_all.jsonl --images ~/sg/imgs --out analysis/objects`

7.3 파일 지도 (insight_fm/)

파일	역할
setup.sh	서버 세팅 전체. STEP=<deps data hf prefetch selftest build>로 단계별 실행
extract_image s.py	Drive zip을 하나씩 받아 라벨된 프레임만 640px로 추출, zip 삭제(이어하기 가능)
build_dataset. py	jsonl → YOLO 폴더. 영상 단위 분할, pilot 샘플링(--pilot-per-class, --pilot-all-classes), 클래스 부분집합(--classes)
encoders.py	FM 인코더 로드(HF / torch hub), feature를 입력의 1/16 격자로 맞춤
fm_yolo.py	fusion-distill 모델과 trainer (Ultralytics 확장)
train_one.py	실험 1개 학습 → 평가 → 지연 측정 → done.json
run_all.py	단계(sanity/pilot/tier2/full/final) 자동 실행, 결과표
export_plain.p y	distill 모델 → 순수 YOLO11s-seg .pt (배포용)
bench_speed.py	추론 속도 종합 측정
analyze_data.p y	개수·분할·크기 분석 (라벨만 사용)
analyze_object s.py	실제 이미지에서 객체별 크기·잘림·선명도·밝기 + 예시 이미지
selftest.py	합성 데이터로 전체 흐름 검증 (CPU 가능)

8. 그동안 겪은 문제와 해결 (같은 실수 방지용)

증상	원인	해결
이미지 추출이 0장 (missing/bad 14826)	서버에 libGL.so.1 없음 → OpenCV import 실패를 코드가 조용히 삼킴	setup.sh 에서 libgl1 libgl2.0-0 설치, 추출 코드가 import 실패 시 즉시 멈추고 실패한 zip은 지우지 않게 수정
S1.zip을 못 찾음	모든 zip을 polygon/ 에서 찾음	jsonl의 z 경로(MyDrive 뒤)로 zip마다 실제 Drive 경로 사용
obstacle/stairs 결과가 뒤바뀜	코드의 클래스 이름이 7=stairs, 8=obstacle로 잘못 적혀 있었음	seg10 모델로 정답(7=obstacle, 8=stairs) 확인 후 수정. 1차 pilot E0는 수정 46초 전에 시작해 done.json 값을 사후 교정(done.json.bak 에 원본)
HF 로그인 실패	폐기된 토큰 입력	토큰은 read -s 로 입력. 로컬 스크립트로 유효성 검사 후 전송
긴 명령 중 Connection closed by remote host	서버가 오래 걸리는 SSH 세션을 끊음	오래 걸리는 작업은 전부 tmux 안에서
터미널에 붙여넣은 명령이 깨짐	화면 폭에서 줄바꿈된 채로 복사됨	긴 명령은 스크립트 파일로 만들어 실행
밤사이 감시가 멈춤	Windows Modern Standby(화면 꺼지면 절전)	전원 연결 시 화면 끄기·절전 "안 함"
2차 서버에서 pip: command not found	conda 없는 이미지	venv 사용 (insight_fm/tools/run_final10.sh 참고)

9. 결론: 무엇이 되고, 안 되고, 왜

	판단	근거
✅ FM fusion (C-RADIOv3)	성능 확실히 옴	최종 test +0.023, 10개 중 9개 클래스 상승. 1차 pilot에서도 fusion 7/7개 상승
✅ GPU 실시간	가능	A5도 end-to-end 44 FPS (E0 75 FPS)
❌ CPU에서 fusion	사실상 불가	CPU 4스레드 2.9초/장 (E0의 약 10배). 노트북 CPU 배포라면 E0(+OpenVINO)만 현실적
❌ FM distill	효과 없음	1차 pilot 8/8개 ±0.008. 최종 학습에서는 제외
⚠️ 평가 수치	실제보다 낮게 나옴	라벨 누락(라바콘·기둥·신호등 뒷면·계단)을 맞게 찾아도 오검출 처리
❌ traffic_light	작은 객체 문제	89%가 16px 미만. 위치는 79% 찾지만 마스크 정밀도에서 손해 → 해상도 상향 필요
❌ stairs	라벨이 틀림	원본 라벨의 상당수가 맨홀·보호판·연석, Polygon 영상의 계단은 라벨 없음 → 재라벨링 필요
❌ scooter	데이터 부족 + 혼동	train 144장, 오토바이·자전거와 혼동, test 94개라 수치도 불안정 → 데이터 추가 필요
⚠️ obstacle	클래스가 넓음	15종 혼합, 가는 기둥형 객체 다수, 라벨 누락 많음
✅ 흐림·조명·카메라	문제 아님	흐린 객체 0.2%, 어두운 프레임 0.4%, 카메라별 차이 작음

10. 다음에 할 일 (우선순위)

1. 노트북(그랩) CPU에서 E0 / A5 실측 (OpenVINO 변환 포함). fusion을 쓸 수 있는지는 이 숫자로 결정된다.
모델은 Drive runs_final10/.
2. stairs 재라벨링(372장, 몇 시간): 계단만 남기고, Polygon 영상 중 계단이 보이는 프레임도 추가 라벨.
3. scooter 데이터 보강: 외부 데이터 또는 추가 라벨링. 지금 양(144장)으로는 어떤 모델도 안 된다.
4. test 라벨 보완(일부라도): 라바콘·신호등 뒷면 등 누락 라벨을 채워야 성능을 제대로 썰 수 있다.
5. traffic_light: imsz 960 또는 1280으로 재학습해 비교(작은 객체 개선 여부).
6. obstacle 세분화(기동형 / 낮은 장애물 / 큰 구조물), traffic_sign 클래스 추가 검토.

11. 계정·자격 증명 (값은 적지 않음)

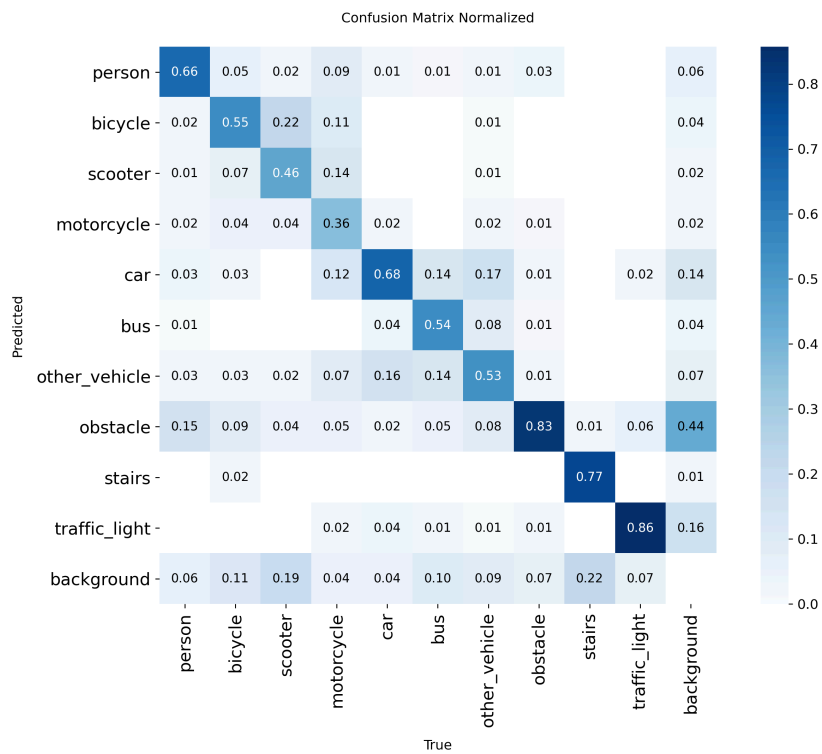
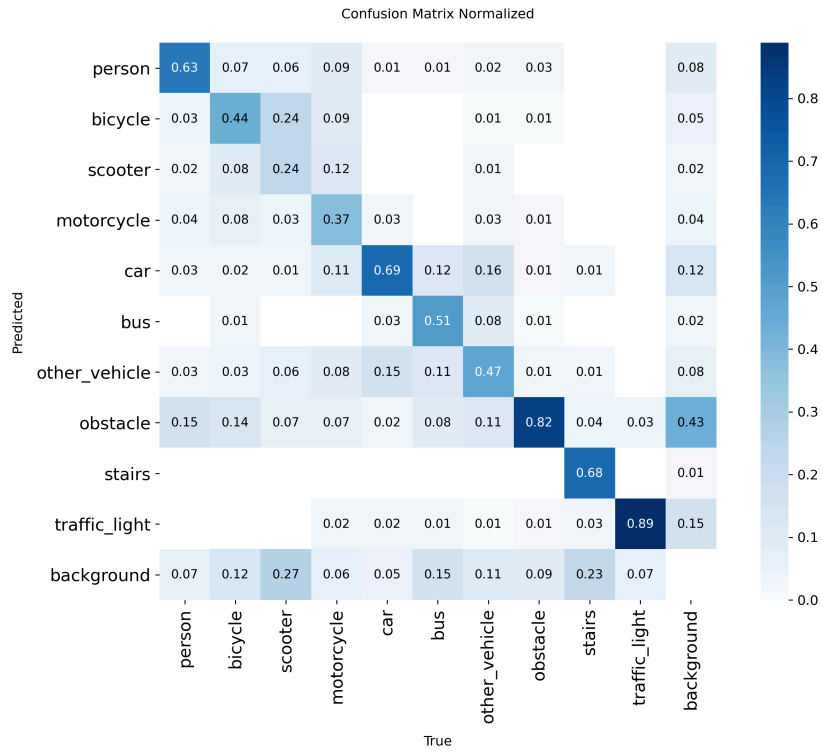
- Hugging Face 토큰: DINOv3(게이트 모델)에만 필요. 대화 중에 한 번 노출된 토큰은 폐기하고 새로 발급함.
- GitHub: 로컬 PC에 `git config --global credential.helper store` 로 저장(평문 `~/.git-credentials`).
- Google Drive: 로컬 PC의 `~/.config/rclone/rclone.conf` (remote 이름 `gdrive:`)를 서버로 복사해서 사용.
- 서버 .pem: 인스턴스마다 새로 발급. 로컬에서는 `~/.ssh/a5000.pem` (권한 600)으로 복사해 사용.

부록 A. 최종 모델 학습 곡선과 혼동 행렬



학습 곡선 (E0, A5)

혼동 행렬은 열(True)이 정답, 행(Predicted)이 예측이다. 각 열의 합이 1이 되도록 정규화했다. 대각선이 높을수록 잘 맞힌 것이고, 맨 아래 행 background는 놓친 비율, 맨 오른쪽 열 background는 오검출이 어느 클래스로 나왔는지다.



부록 B. 추론 속도 상세

모델	방식	인코더	파라미터 (YOLO+FM)	GFLOPs	GPU FP16 (ms)	E0 대비	GPU FP32 (ms)	GPU bs8 (img/s)	실제 이미지 end-to-end (ms / FPS)	CPU 4 스레드 (ms)	CPU 8 스레드 (ms)
A5	fusion	c-radio_v3-b	108.3M	408.0	21.13	1.91x	20.21	93.1	22.51 / 44.4	2866.9	1615.2
E0	none	-	10.1M	32.9	11.08	1.00x	10.44	555.4	13.38 / 74.7	293.6	194.9

fusion 모델의 FM 비중 (GPU FP16)

- A5: 전체 21.13ms 중 FM 10.07ms (47.7%), CPU 4스레드에서는 FM만 2539.1ms

end-to-end는 실제 test 이미지에서 전처리 + 추론 + 후처리(NMS, 마스크) 중앙값. CPU 수치는 서버 CPU의 스레드 수를 제한해 낸 값으로, 노트북 CPU의 실제 속도와는 다를 수 있음.

부록 C. 오류 분석 상세: E0 (YOLO11s-seg)

test 13,721장. 기준: confidence ≥ 0.25 , 같은 클래스 박스 IoU ≥ 0.5 면 '찾음'. AP는 모든 confidence를 훑지만, 여기서는 한 지점에서 찾은 것 / 놓친 것 / 잘못 찾은 것을 센다.

1. 클래스별 요약

클래스	정답 객체	찾음(recall %)	오검출 (FP)	정답 1개당 FP	precision %	FP 중 다른 클래스 정답과 겹침 %
person	7,174	5,373 (74.9)	1,537	0.21	77.8	13.8
bicycle	1,600	922 (57.6)	680	0.42	57.6	25.0
scooter	94	34 (36.2)	312	3.32	9.8	49.0
motorcycle	1,394	924 (66.3)	799	0.57	53.6	33.3
car	22,566	18,803 (83.3)	6,815	0.30	73.4	13.9
bus	1,978	1,087 (55.0)	528	0.27	67.3	29.2
other_vehicle	5,317	3,075 (57.8)	1,682	0.32	64.6	36.9
obstacle	43,540	28,298 (65.0)	14,844	0.34	65.6	1.3
stairs	79	34 (43.0)	80	1.01	29.8	1.2
traffic_light	4,193	3,295 (78.6)	3,787	0.90	46.5	0.6

2. 크기별 recall (박스 짧은 변, 640px 기준)

클래스	0~8px	8~16px	16~32px	32~64px	64px 이상
person	25.8% (n=555)	68.4% (n=2,570)	83.0% (n=2,493)	88.6% (n=1,124)	94.2% (n=432)
bicycle	0.0% (n=122)	24.1% (n=253)	56.1% (n=487)	78.1% (n=433)	82.0% (n=305)
scooter	0.0% (n=11)	10.5% (n=19)	30.8% (n=26)	70.4% (n=27)	45.5% (n=11)
motorcycle	0.0% (n=44)	33.3% (n=192)	67.0% (n=439)	76.8% (n=379)	80.9% (n=340)
car	16.3% (n=1,175)	65.6% (n=3,792)	87.2% (n=7,515)	93.2% (n=5,530)	97.0% (n=4,554)
bus	5.8% (n=139)	18.3% (n=312)	49.5% (n=618)	75.7% (n=497)	82.5% (n=412)
other_vehicle	3.2% (n=311)	18.3% (n=677)	53.4% (n=1,571)	70.3% (n=1,358)	81.9% (n=1,400)
obstacle	41.4% (n=7,452)	64.4% (n=15,671)	73.2% (n=12,395)	75.6% (n=5,847)	74.8% (n=2,175)
stairs	0.0% (n=6)	25.0% (n=8)	33.3% (n=15)	28.6% (n=14)	63.9% (n=36)
traffic_light	75.4% (n=2,707)	82.4% (n=1,114)	89.1% (n=322)	97.9% (n=47)	66.7% (n=3)

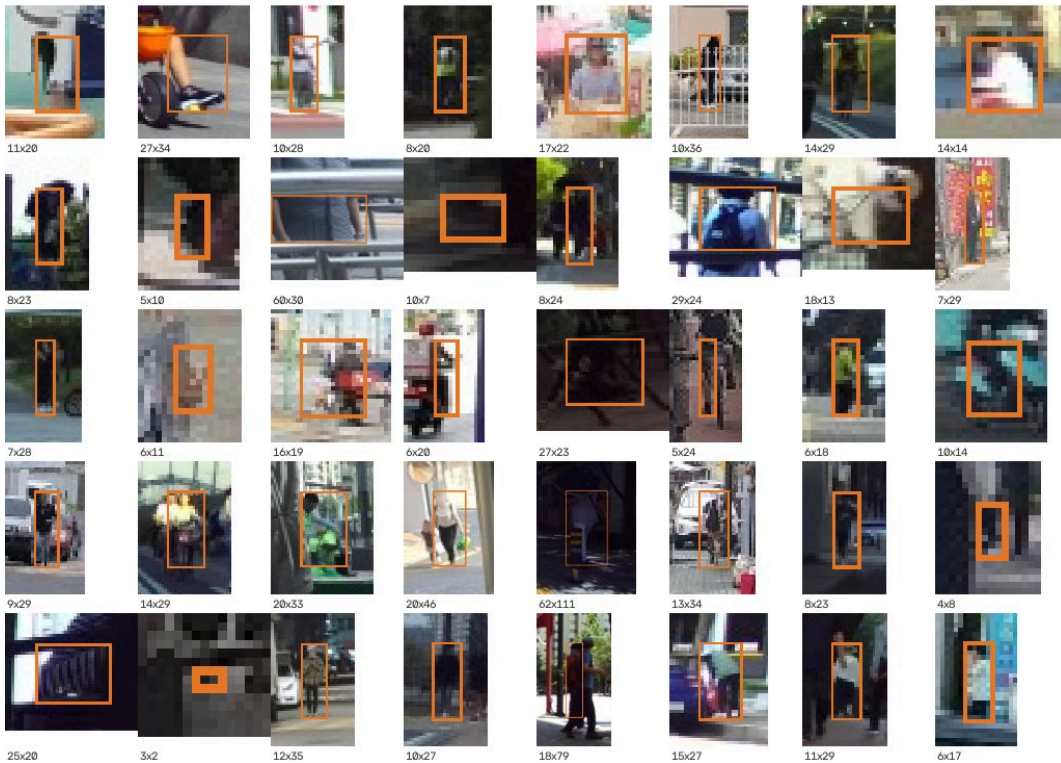
3. 잘림 · 밝기 · 카메라별 recall

클래스	잘리지 않음	잘림	밝음(≥50)	어두움(<50)	스마트폰	ZED
person	74.6% (n=6,741)	80.1% (n=433)	75.4% (n=5,340)	73.3% (n=1,834)	76.9% (n=3,556)	73.0% (n=3,618)
bicycle	55.2% (n=1,394)	74.3% (n=206)	58.4% (n=1,191)	55.3% (n=409)	58.9% (n=880)	56.1% (n=720)
scooter	35.2% (n=88)	50.0% (n=6)	34.5% (n=87)	57.1% (n=7)	36.2% (n=80)	35.7% (n=14)
motorcycle	66.2% (n=1,223)	66.7% (n=171)	67.2% (n=1,113)	62.6% (n=281)	68.4% (n=974)	61.4% (n=420)
car	81.2% (n=18,805)	93.9% (n=3,761)	84.6% (n=19,605)	74.9% (n=2,961)	85.4% (n=15,387)	78.8% (n=7,179)
bus	52.0% (n=1,593)	67.0% (n=385)	56.8% (n=1,655)	45.5% (n=323)	61.0% (n=898)	49.9% (n=1,080)
other_vehicle	54.6% (n=4,348)	72.5% (n=969)	61.1% (n=4,374)	42.8% (n=943)	63.1% (n=3,257)	49.5% (n=2,060)
obstacle	59.1% (n=30,725)	79.2% (n=12,815)	66.4% (n=33,516)	60.3% (n=10,024)	64.0% (n=22,971)	66.1% (n=20,569)
stairs	25.8% (n=31)	54.2% (n=48)	43.0% (n=79)	-	43.0% (n=79)	-
traffic_light	78.3% (n=3,923)	83.3% (n=270)	79.8% (n=2,827)	76.1% (n=1,366)	81.4% (n=1,326)	77.3% (n=2,867)

4. 예시 (파란 박스. 놓친 것 = 정답 위치, 오검출 = 예측 위치, c = confidence, ~클래스 = 겹친 다른 정답)

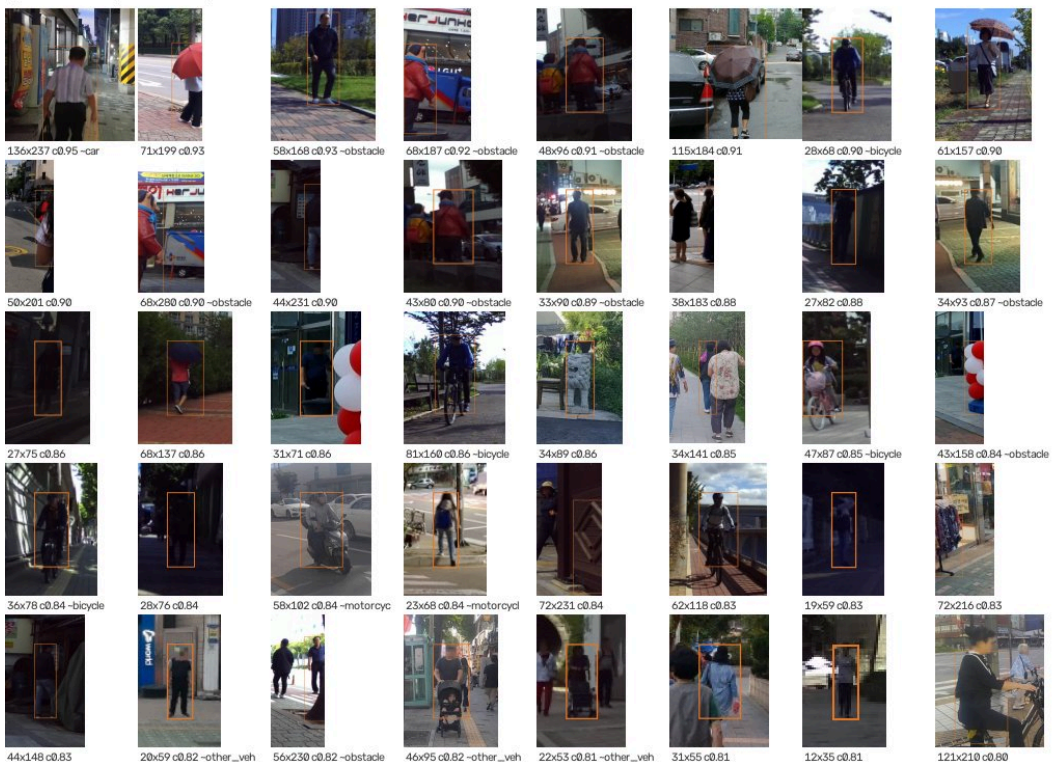
person: 놓친 객체 (1,801개 중 무작위 40)

person: missed, random 40 of 1801



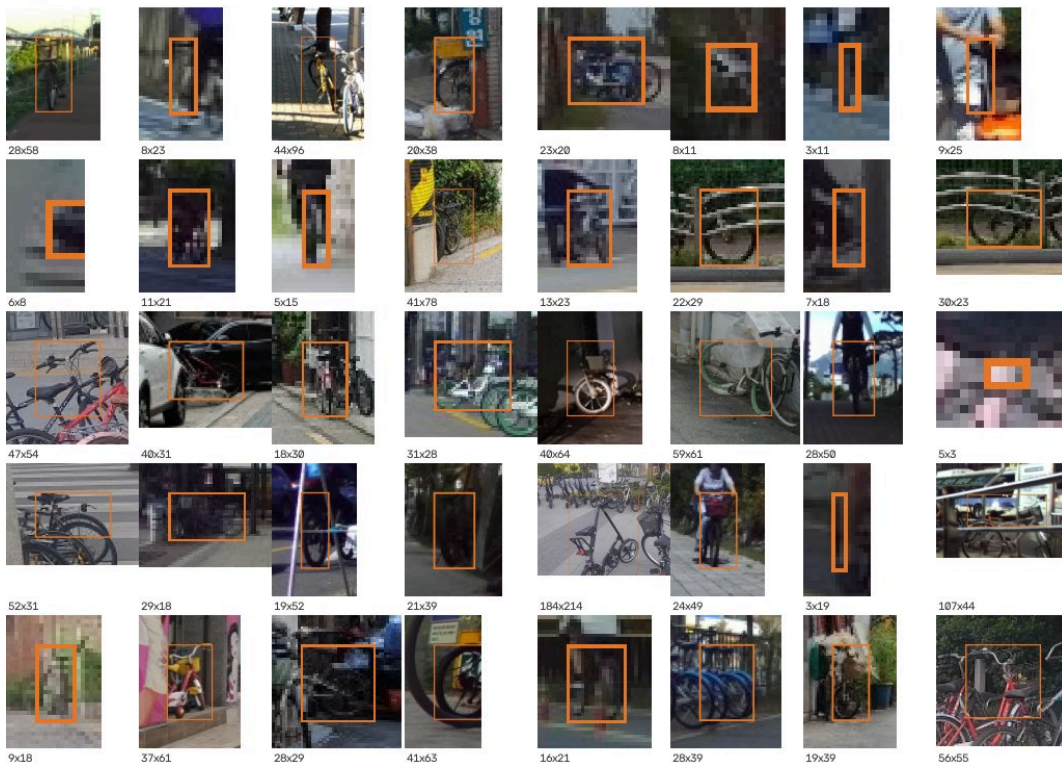
person: 오검출 (confidence 상위 40 / 1,537개)

person: false positives, top 40 conf of 1537



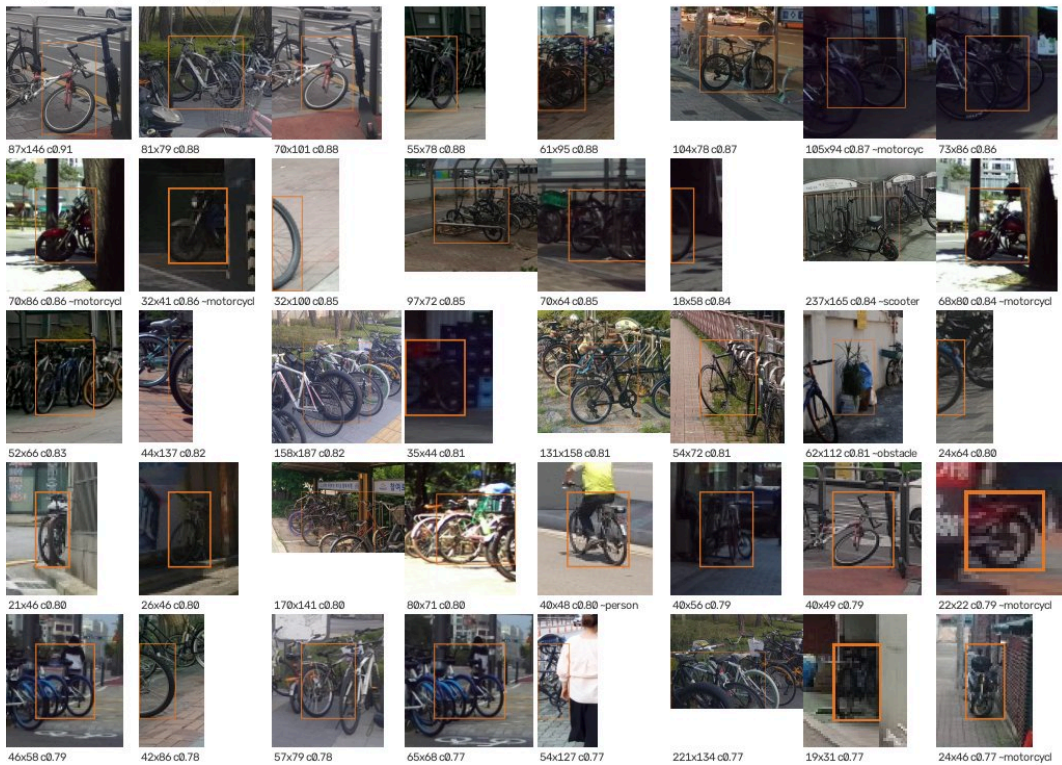
bicycle: 놓친 객체 (678개 중 무작위 40)

bicycle: missed, random 40 of 678



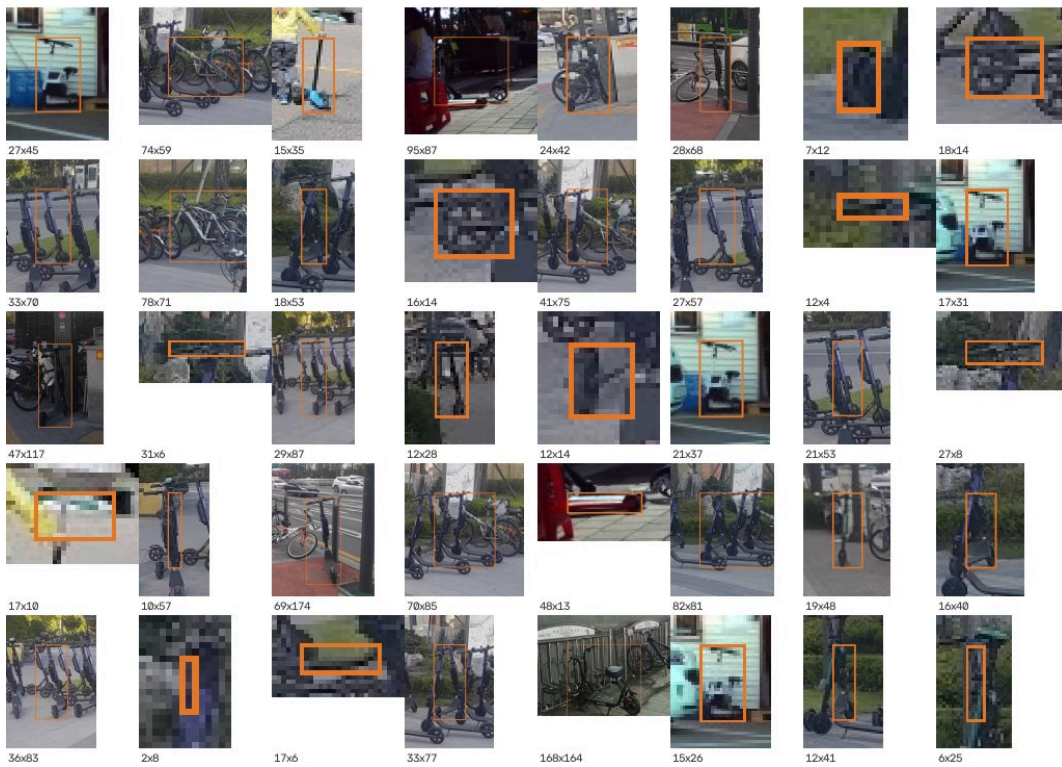
bicycle: 오검출 (confidence 상위 40 / 680개)

bicycle: false positives, top 40 conf of 680



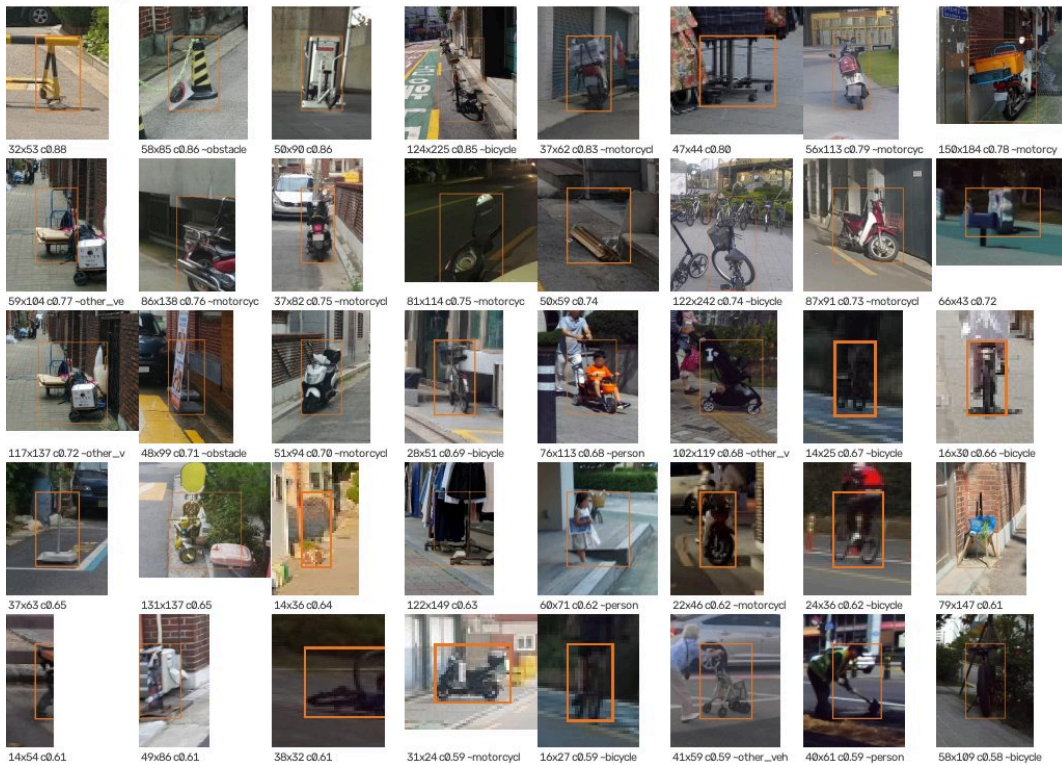
scooter: 놓친 객체 (60개 중 무작위 40)

scooter: missed, random 40 of 60



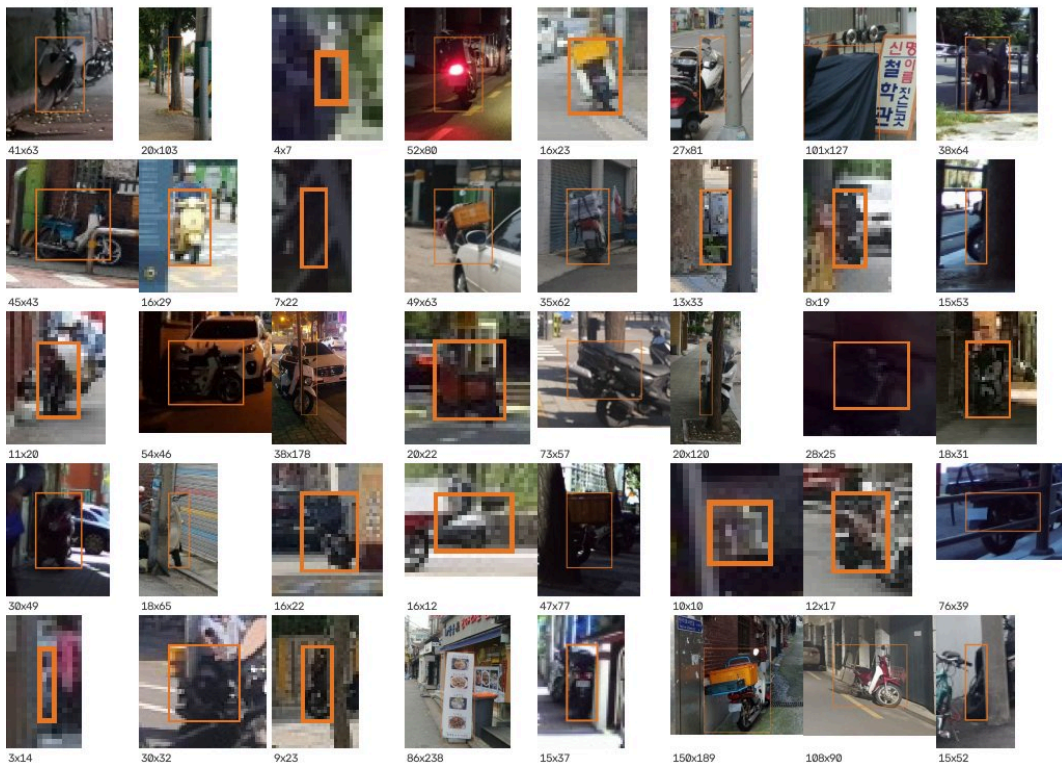
scooter: 오검출 (confidence 상위 40 / 312개)

scooter: false positives, top 40 conf of 312



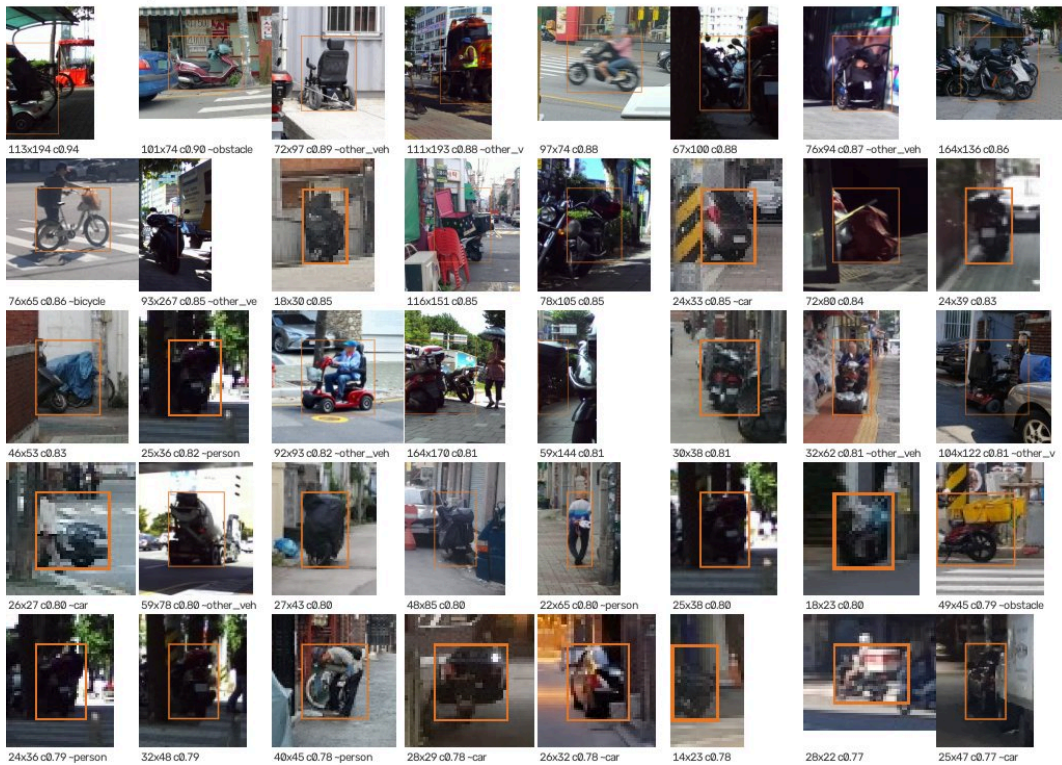
motorcycle: 놓친 객체 (470개 중 무작위 40)

motorcycle: missed, random 40 of 470



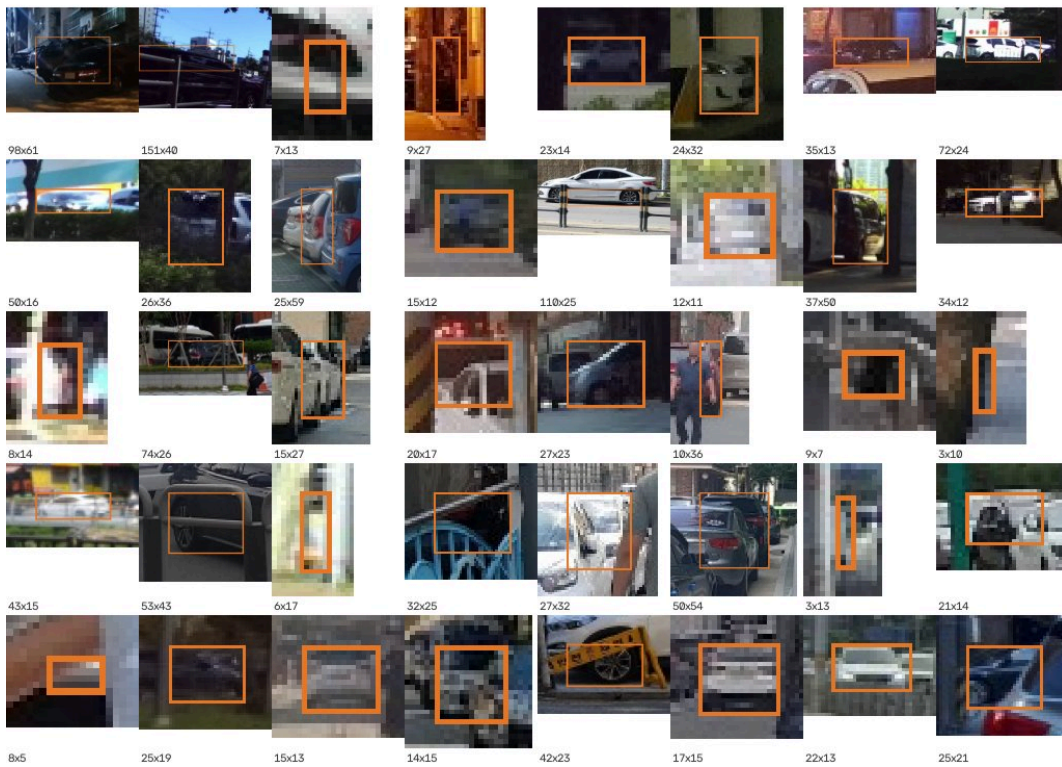
motorcycle: 오검출 (confidence 상위 40 / 799개)

motorcycle: false positives, top 40 conf of 799



car: 놓친 객체 (3,763개 중 무작위 40)

car: missed, random 40 of 3763



car: 오검출 (confidence 상위 40 / 6,815개)

car: false positives, top 40 conf of 6815



bus: 놓친 객체 (891개 중 무작위 40)

bus: missed, random 40 of 891



bus: 오검출 (confidence 상위 40 / 528개)

bus: false positives, top 40 conf of 528



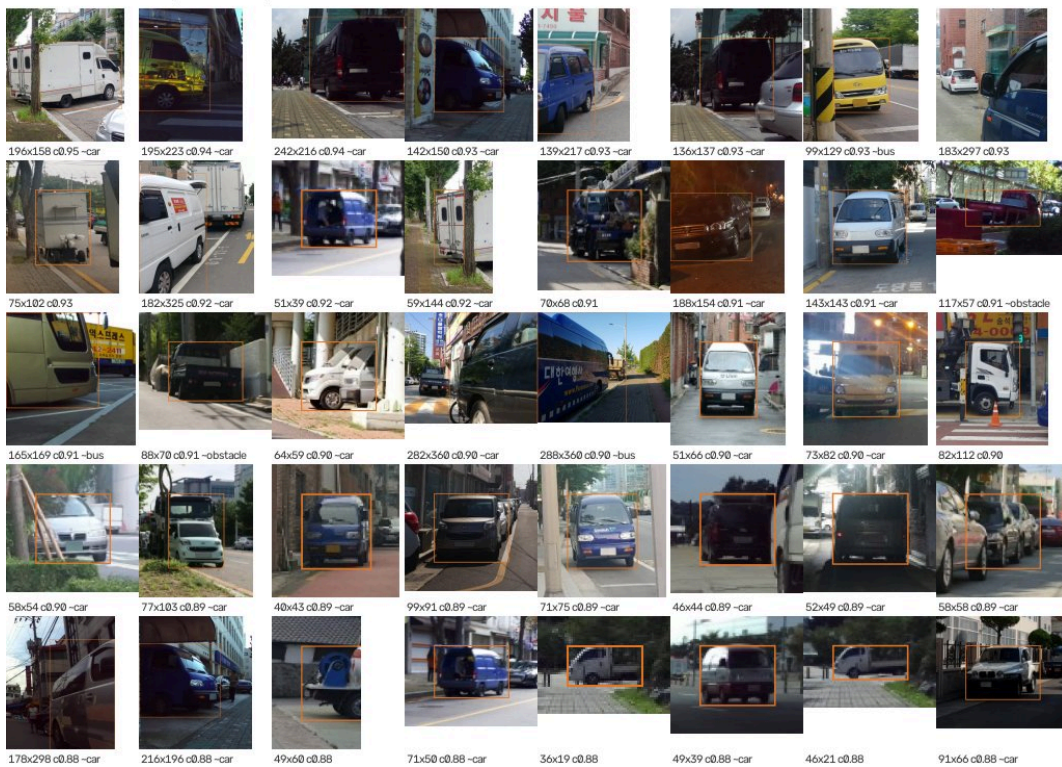
other_vehicle: 놓친 객체 (2,242개 중 무작위 40)

other_vehicle: missed, random 40 of 2242



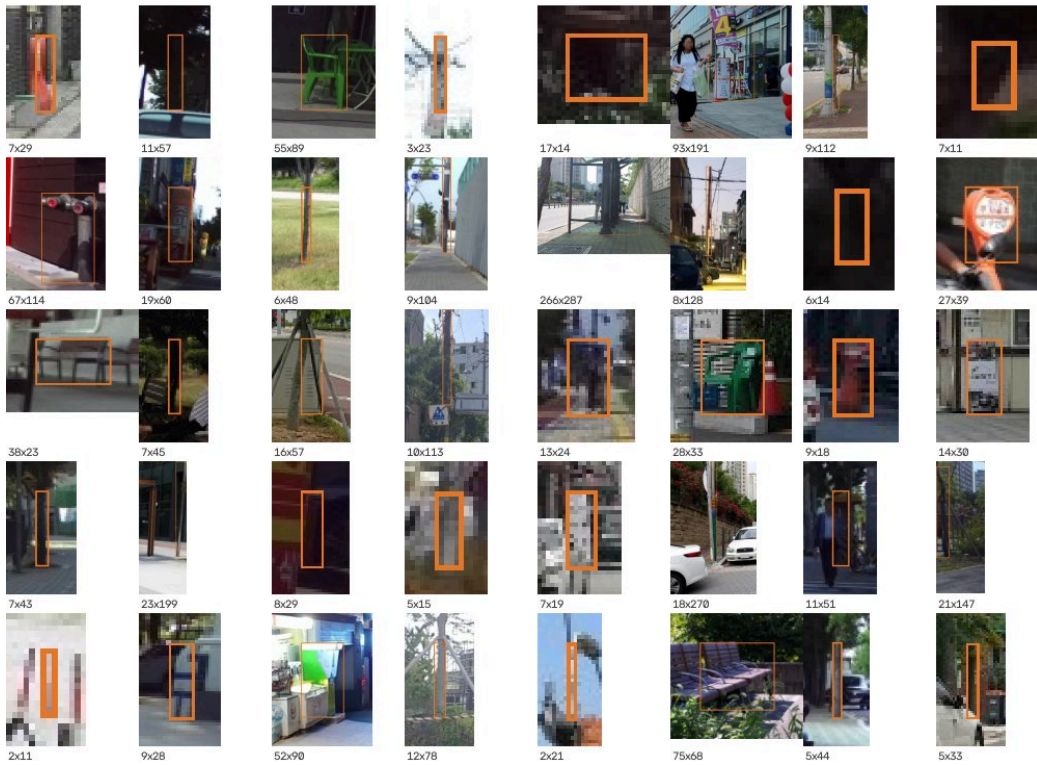
other_vehicle: 오검출 (confidence 상위 40 / 1,682개)

other_vehicle: false positives, top 40 conf of 1682



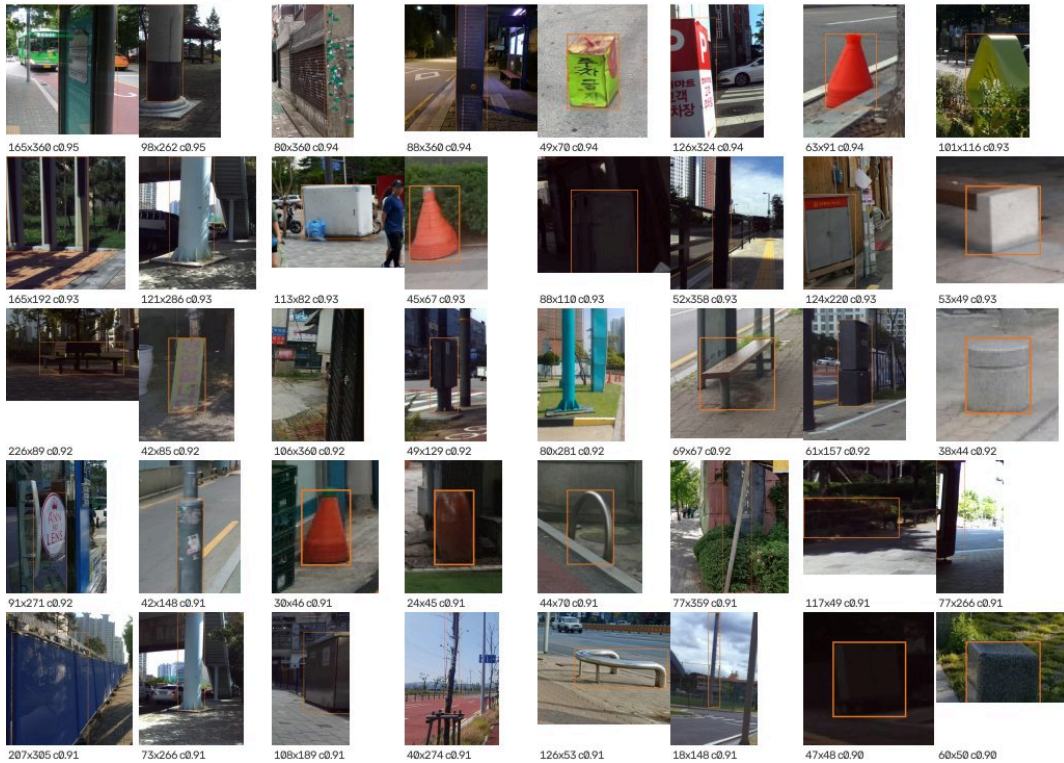
obstacle: 농친 객체 (15,242개 중 무작위 40)

obstacle: missed, random 40 of 15242



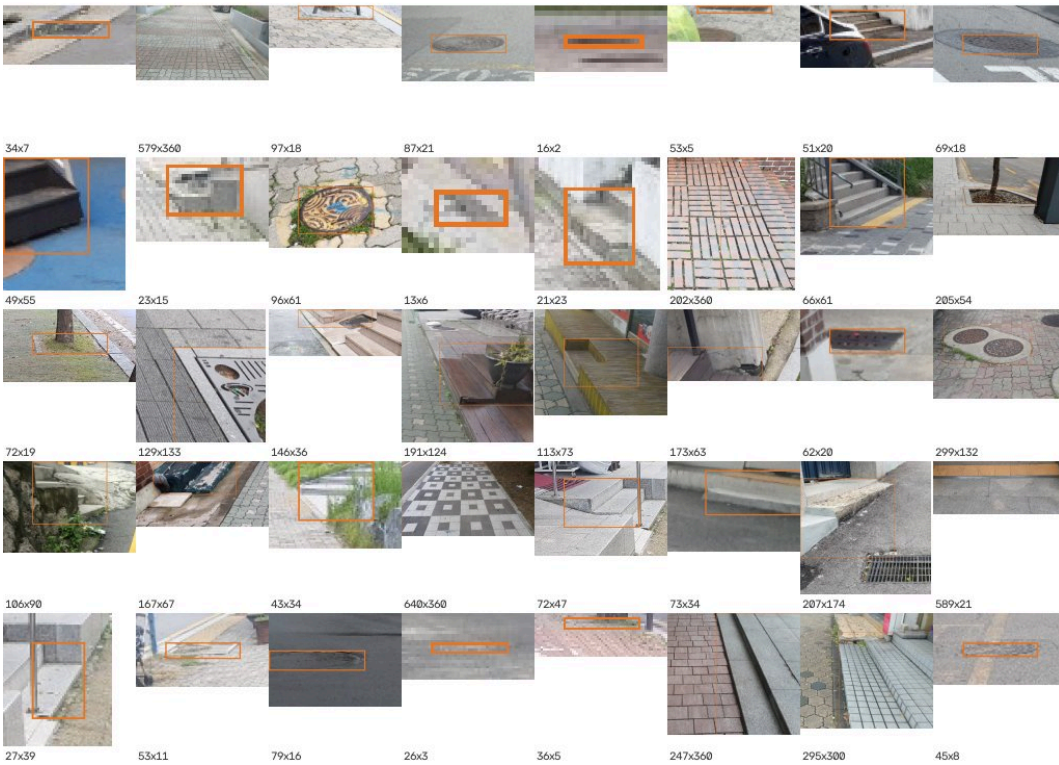
obstacle: 오검출 (confidence 상위 40 / 14,844개)

obstacle: false positives, top 40 conf of 14844



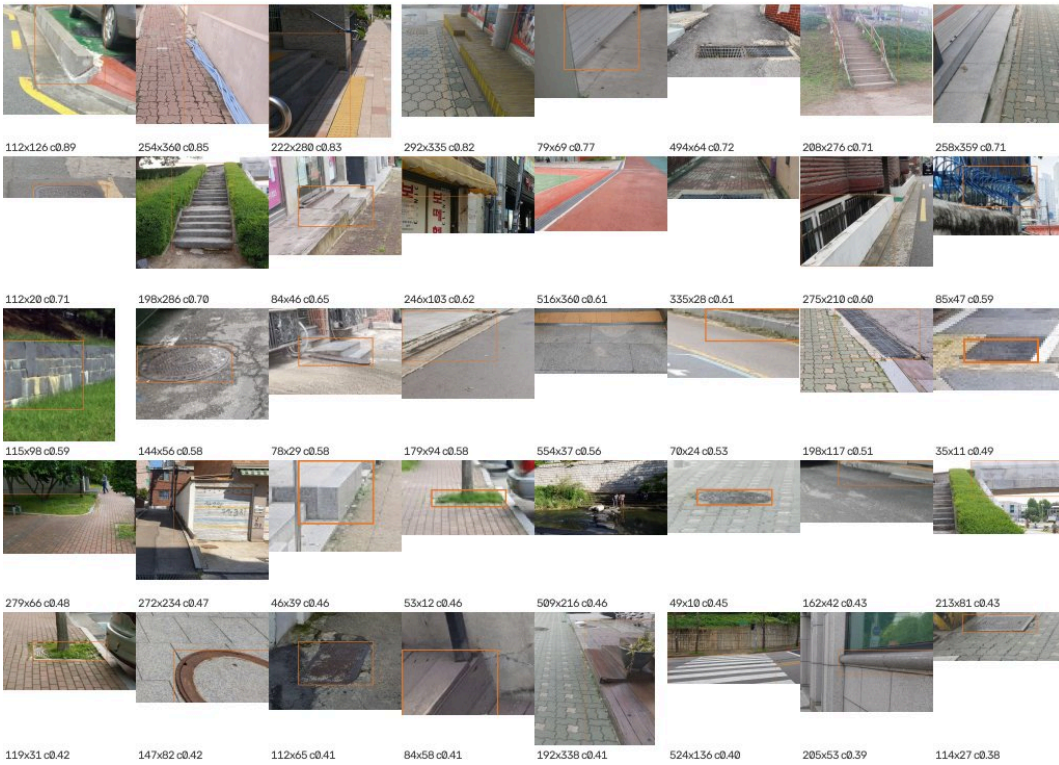
stairs: 놓친 객체 (45개 중 무작위 40)

stairs: missed, random 40 of 45



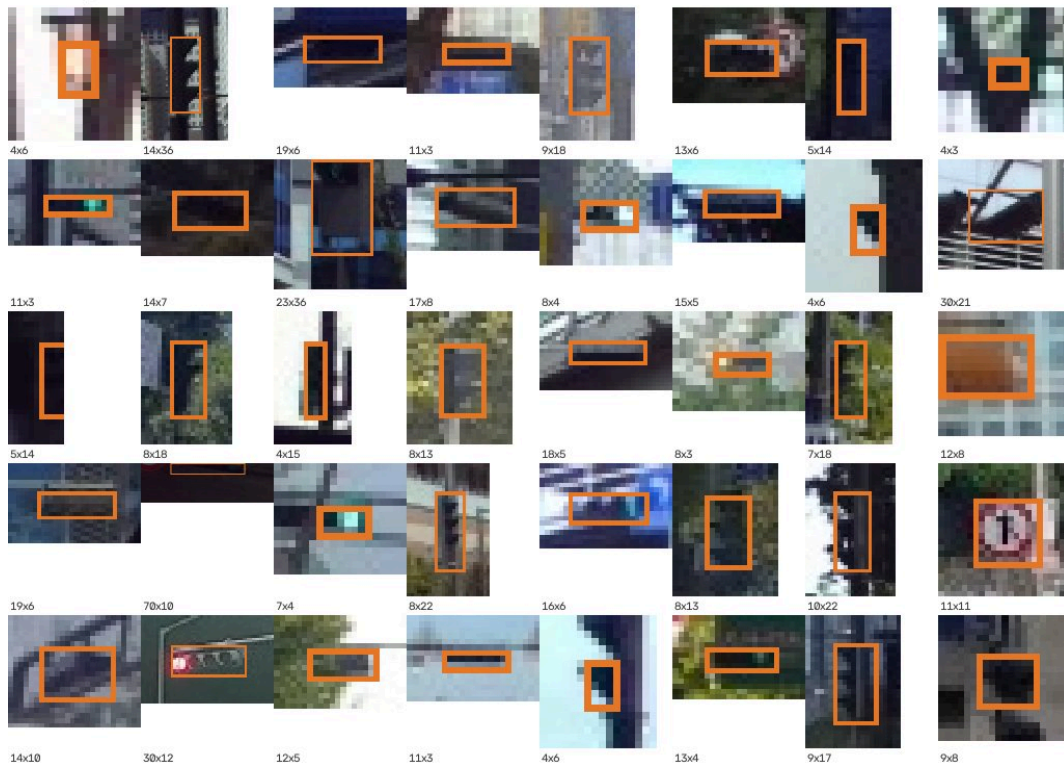
stairs: 오검출 (confidence 상위 40 / 80개)

stairs: false positives, top 40 conf of 80



traffic_light: 놓친 객체 (898개 중 무작위 40)

traffic_light: missed, random 40 of 898



traffic_light: 오검출 (confidence 상위 40 / 3,787개)

traffic_light: false positives, top 40 conf of 3787



테스트 이미지 중 클래스가 하나라도 있는 사진 수: person 4,054, bicycle 1,031, scooter 33, motorcycle 1,025, car 8,307, bus 1,138, other_vehicle 3,286, obstacle 12,123, stairs 72, traffic_light 1,680

부록 D. 오류 분석 상세: A5 (+C-RADIOv3)

test 13,721장. 기준: confidence ≥ 0.25 , 같은 클래스 박스 IoU ≥ 0.5 면 '찾음'. AP는 모든 confidence를 흘지만, 여기서는 한 지점에서 찾은 것 / 놓친 것 / 잘못 찾은 것을 센다.

1. 클래스별 요약

클래스	정답 객체	찾음(recall %)	오검출 (FP)	정답 1개당 FP	precision %	FP 중 다른 클래스 정답과 겹침 %
person	7,174	5,620 (78.3)	1,634	0.23	77.5	11.9
bicycle	1,600	933 (58.3)	569	0.36	62.1	17.6
scooter	94	32 (34.0)	213	2.27	13.1	46.5
motorcycle	1,394	933 (66.9)	304	0.22	75.4	28.0
car	22,566	18,963 (84.0)	6,581	0.29	74.2	12.2
bus	1,978	1,302 (65.8)	954	0.48	57.7	20.2
other_vehicle	5,317	3,178 (59.8)	1,385	0.26	69.6	33.2
obstacle	43,540	28,689 (65.9)	13,887	0.32	67.4	1.1
stairs	79	29 (36.7)	74	0.94	28.2	2.7
traffic_light	4,193	3,230 (77.0)	3,616	0.86	47.2	0.6

2. 크기별 recall (박스 짧은 변, 640px 기준)

클래스	0~8px	8~16px	16~32px	32~64px	64px 이상
person	21.8% (n=555)	70.7% (n=2,570)	87.8% (n=2,493)	95.2% (n=1,124)	97.5% (n=432)
bicycle	0.8% (n=122)	24.5% (n=253)	53.2% (n=487)	79.9% (n=433)	86.9% (n=305)
scooter	0.0% (n=11)	5.3% (n=19)	34.6% (n=26)	59.3% (n=27)	54.5% (n=11)
motorcycle	0.0% (n=44)	19.8% (n=192)	63.6% (n=439)	81.5% (n=379)	90.3% (n=340)
car	13.8% (n=1,175)	65.4% (n=3,792)	88.0% (n=7,515)	94.7% (n=5,530)	98.2% (n=4,554)
bus	10.8% (n=139)	27.6% (n=312)	62.8% (n=618)	87.5% (n=497)	91.7% (n=412)
other_vehicle	0.6% (n=311)	10.6% (n=677)	48.1% (n=1,571)	76.3% (n=1,358)	93.8% (n=1,400)
obstacle	35.8% (n=7,452)	63.2% (n=15,671)	77.0% (n=12,395)	81.6% (n=5,847)	82.7% (n=2,175)
stairs	0.0% (n=6)	0.0% (n=8)	6.7% (n=15)	50.0% (n=14)	58.3% (n=36)
traffic_light	72.5% (n=2,707)	83.8% (n=1,114)	89.4% (n=322)	95.7% (n=47)	66.7% (n=3)

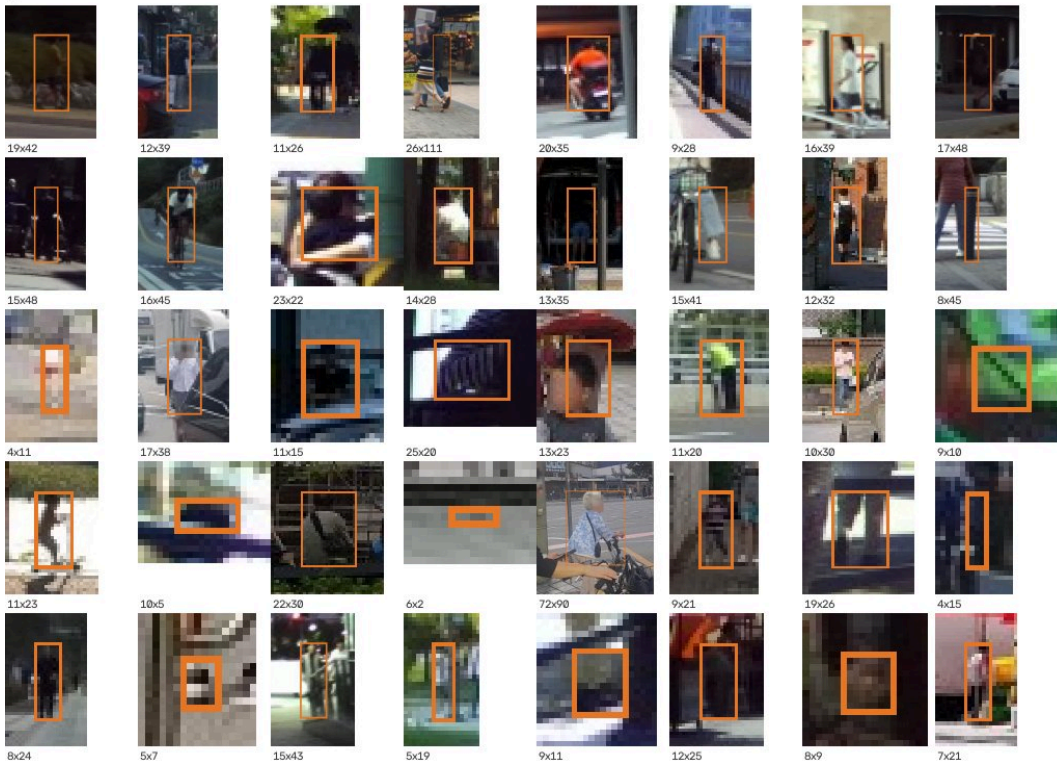
3. 잘림 · 밝기 · 카메라별 recall

클래스	잘리지 않음	잘림	밝음(≥50)	어두움(<50)	스마트폰	ZED
person	77.6% (n=6,741)	90.1% (n=433)	78.7% (n=5,340)	77.2% (n=1,834)	79.2% (n=3,556)	77.4% (n=3,618)
bicycle	55.6% (n=1,394)	76.7% (n=206)	59.9% (n=1,191)	53.5% (n=409)	60.3% (n=880)	55.8% (n=720)
scooter	31.8% (n=88)	66.7% (n=6)	33.3% (n=87)	42.9% (n=7)	31.2% (n=80)	50.0% (n=14)
motorcycle	64.6% (n=1,223)	83.6% (n=171)	67.4% (n=1,113)	65.1% (n=281)	69.0% (n=974)	62.1% (n=420)
car	81.8% (n=18,805)	95.3% (n=3,761)	85.1% (n=19,605)	77.1% (n=2,961)	86.2% (n=15,387)	79.5% (n=7,179)
bus	62.3% (n=1,593)	80.5% (n=385)	66.0% (n=1,655)	64.7% (n=323)	70.7% (n=898)	61.8% (n=1,080)
other_vehicle	54.3% (n=4,348)	84.2% (n=969)	62.8% (n=4,374)	45.7% (n=943)	66.7% (n=3,257)	48.8% (n=2,060)
obstacle	58.9% (n=30,725)	82.6% (n=12,815)	67.6% (n=33,516)	60.1% (n=10,024)	66.1% (n=22,971)	65.7% (n=20,569)
stairs	12.9% (n=31)	52.1% (n=48)	36.7% (n=79)	-	36.7% (n=79)	-
traffic_light	76.4% (n=3,923)	85.9% (n=270)	78.2% (n=2,827)	74.6% (n=1,366)	80.8% (n=1,326)	75.3% (n=2,867)

4. 예시 (파란 박스. 놓친 것 = 정답 위치, 오검출 = 예측 위치, c = confidence, ~클래스 = 겹친 다른 정답)

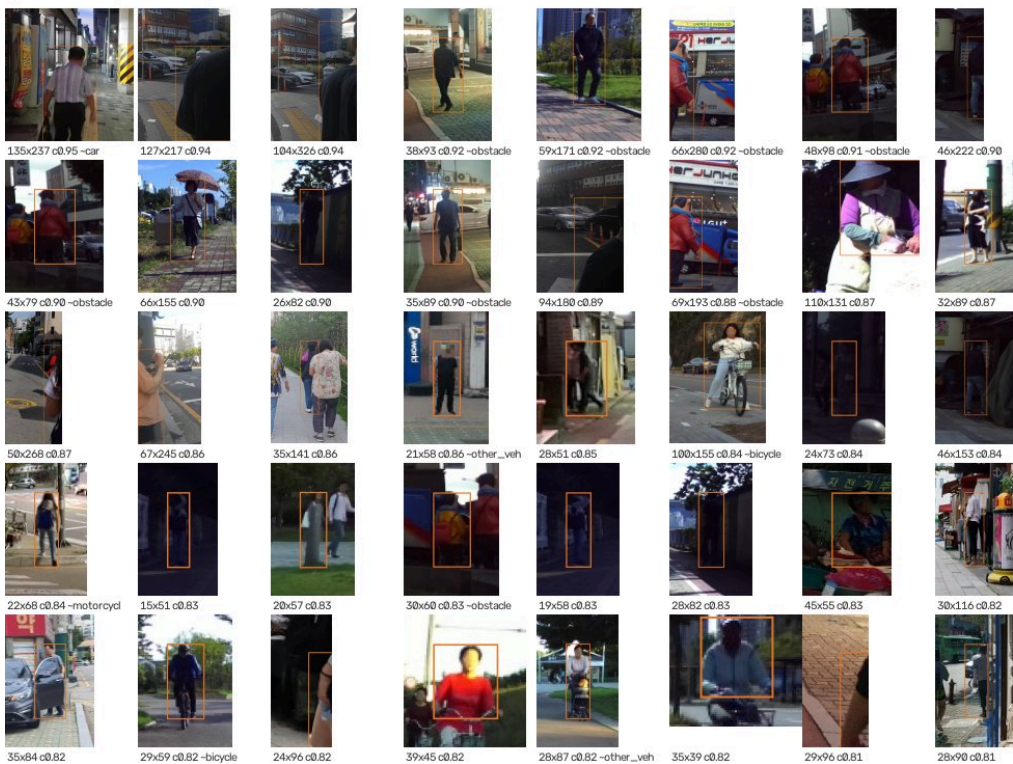
person: 놓친 객체 (1,554개 중 무작위 40)

person: missed, random 40 of 1554



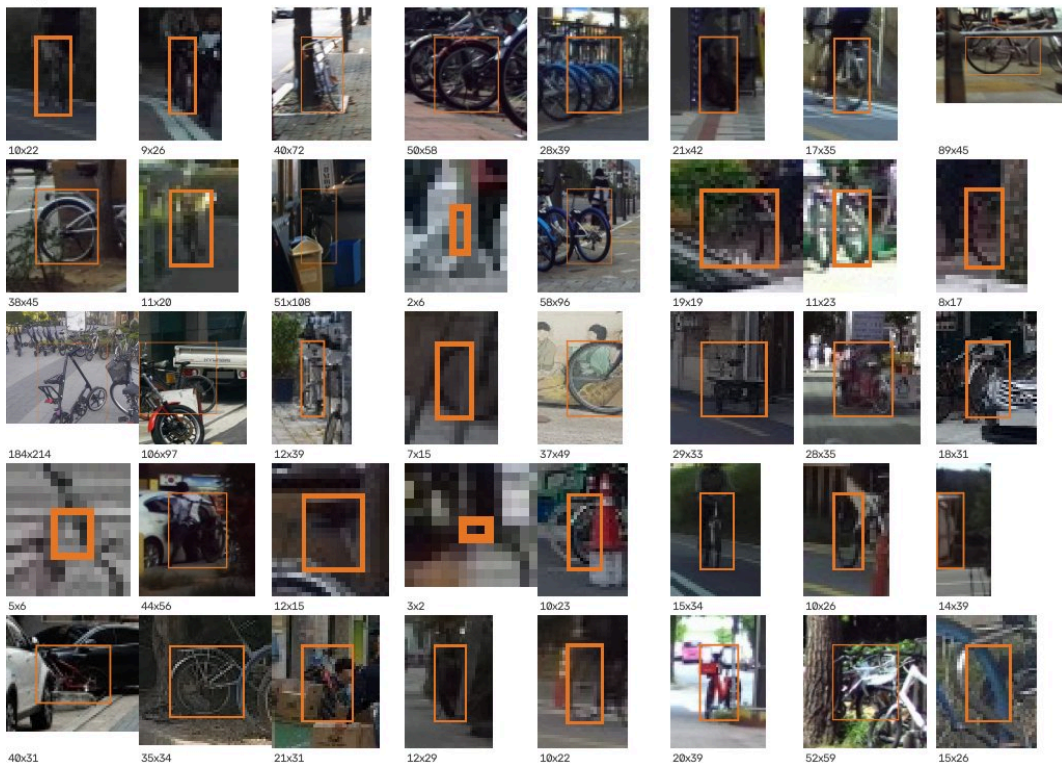
person: 오검출 (confidence 상위 40 / 1,634개)

person: false positives, top 40 conf of 1634



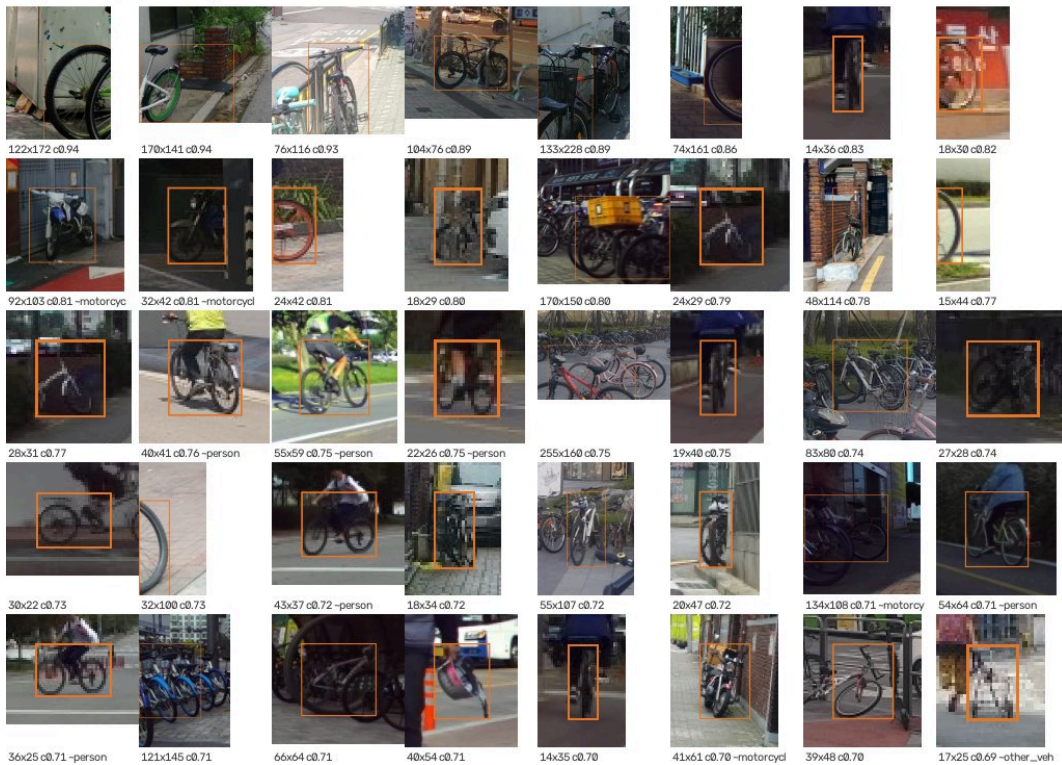
bicycle: 놓친 객체 (667개 중 무작위 40)

bicycle: missed, random 40 of 667



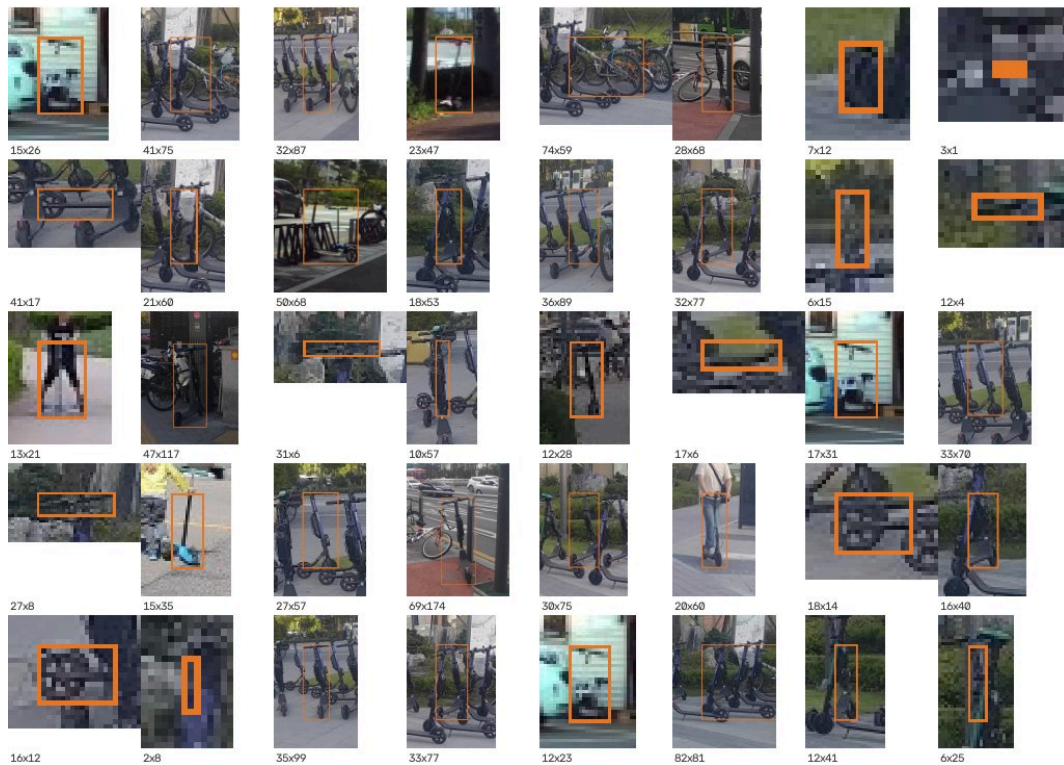
bicycle: 오검출 (confidence 상위 40 / 569개)

bicycle: false positives, top 40 conf of 569



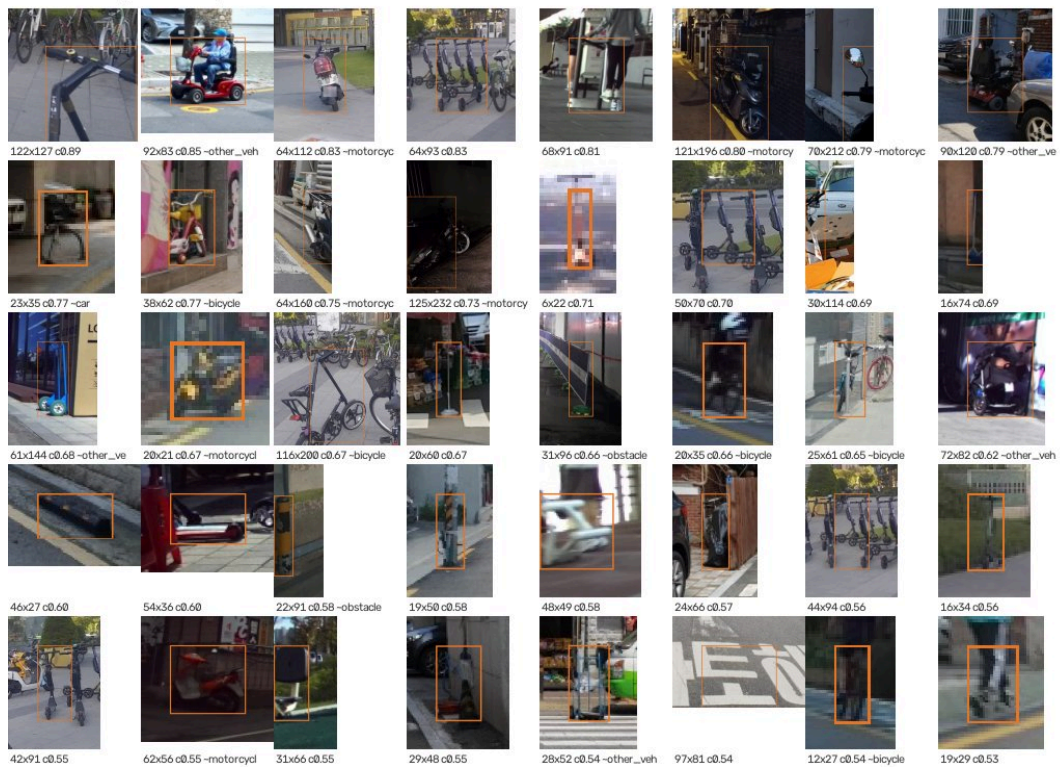
scooter: 놓친 객체 (62개 중 무작위 40)

scooter: missed, random 40 of 62



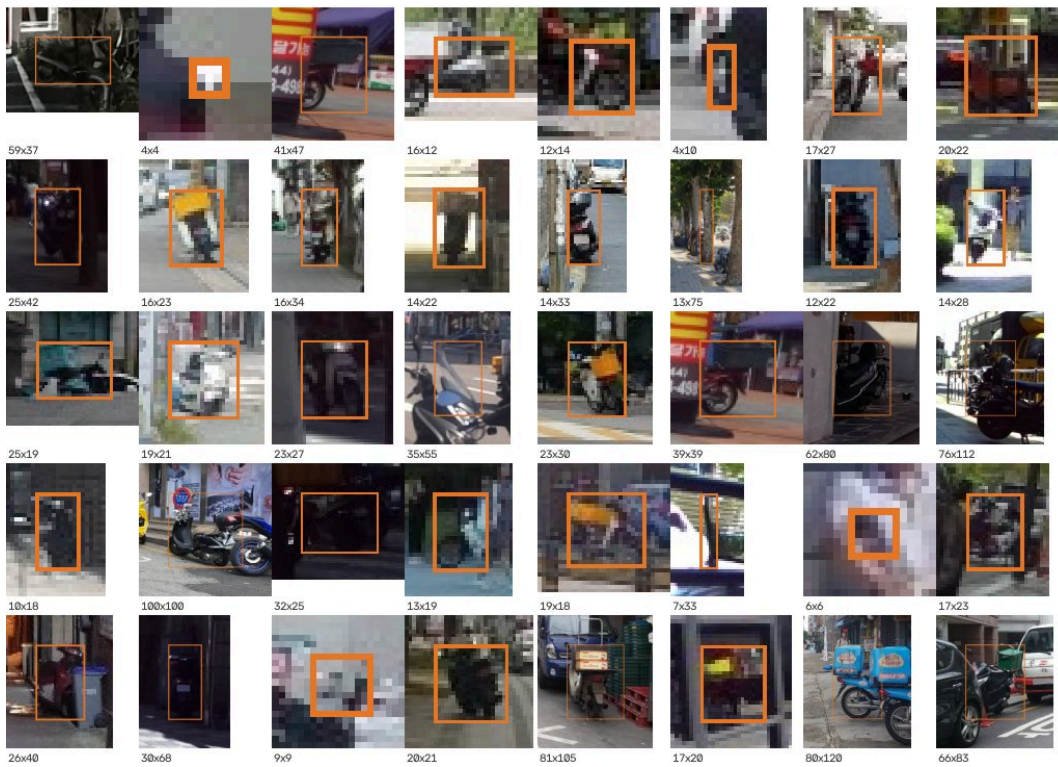
scooter: 오검출 (confidence 상위 40 / 213개)

scooter: false positives, top 40 conf of 213



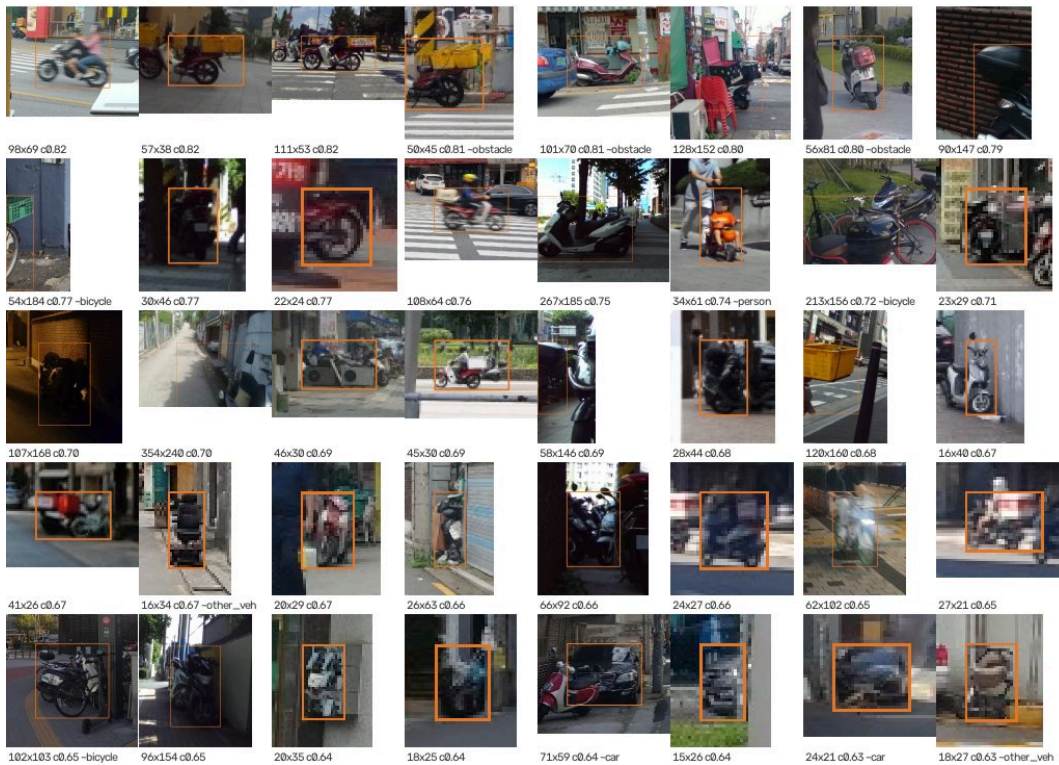
motorcycle: 놓친 객체 (461개 중 무작위 40)

motorcycle: missed, random 40 of 461



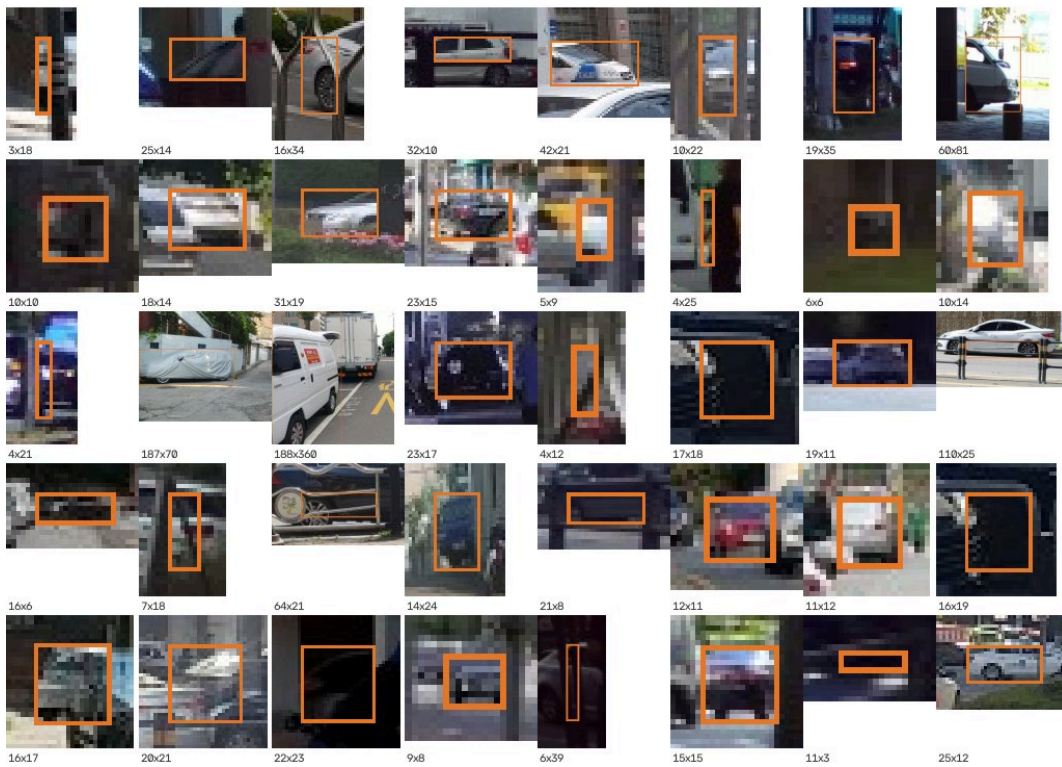
motorcycle: 오검출 (confidence 상위 40 / 304개)

motorcycle: false positives, top 40 conf of 304



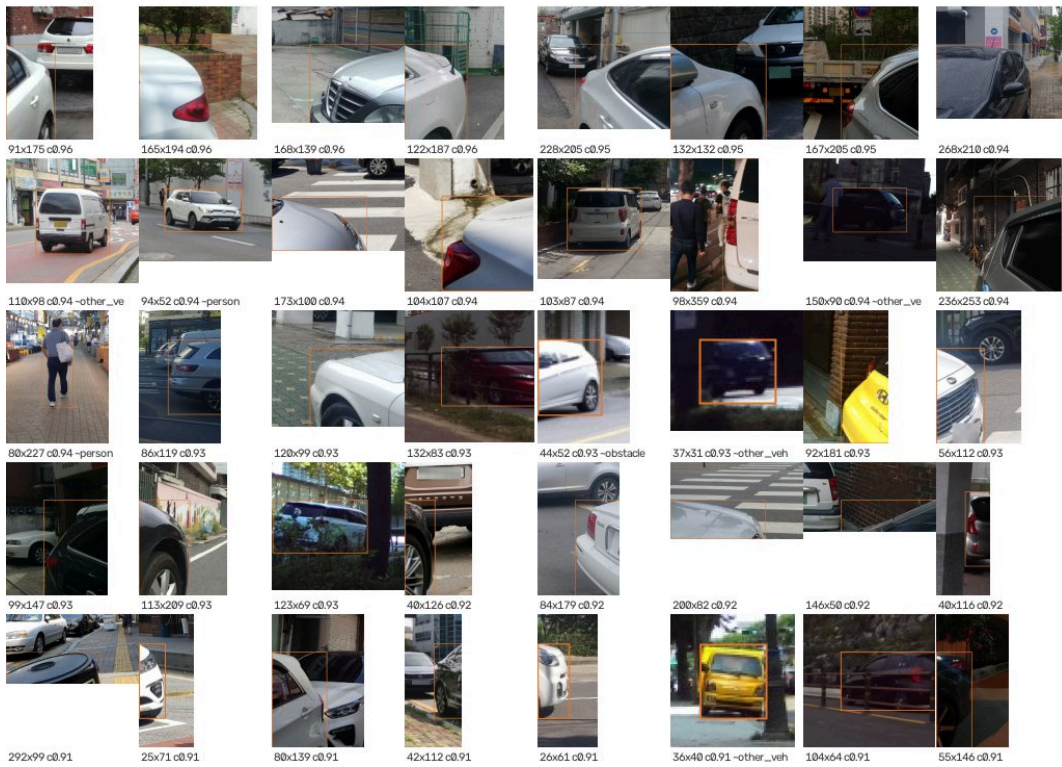
car: 놓친 객체 (3,603개 중 무작위 40)

car: missed, random 40 of 3603



car: 오검출 (confidence 상위 40 / 6,581개)

car: false positives, top 40 conf of 6581



bus: 놓친 객체 (676개 중 무작위 40)

bus: missed, random 40 of 676



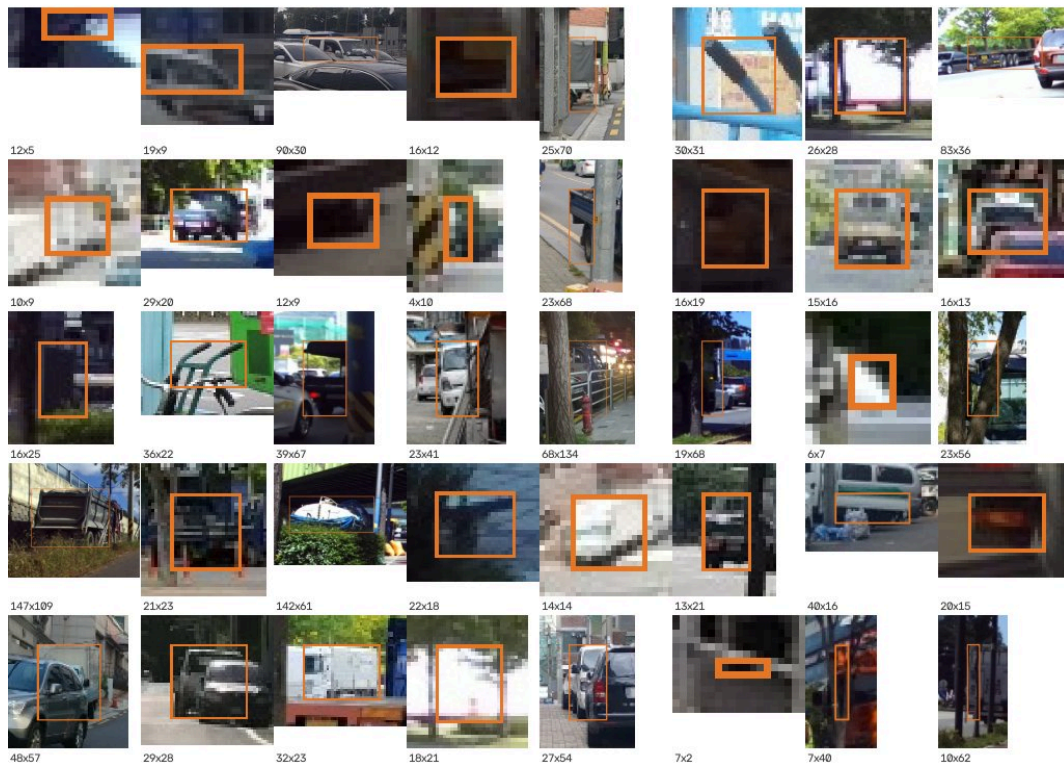
bus: 오검출 (confidence 상위 40 / 954개)

bus: false positives, top 40 conf of 954



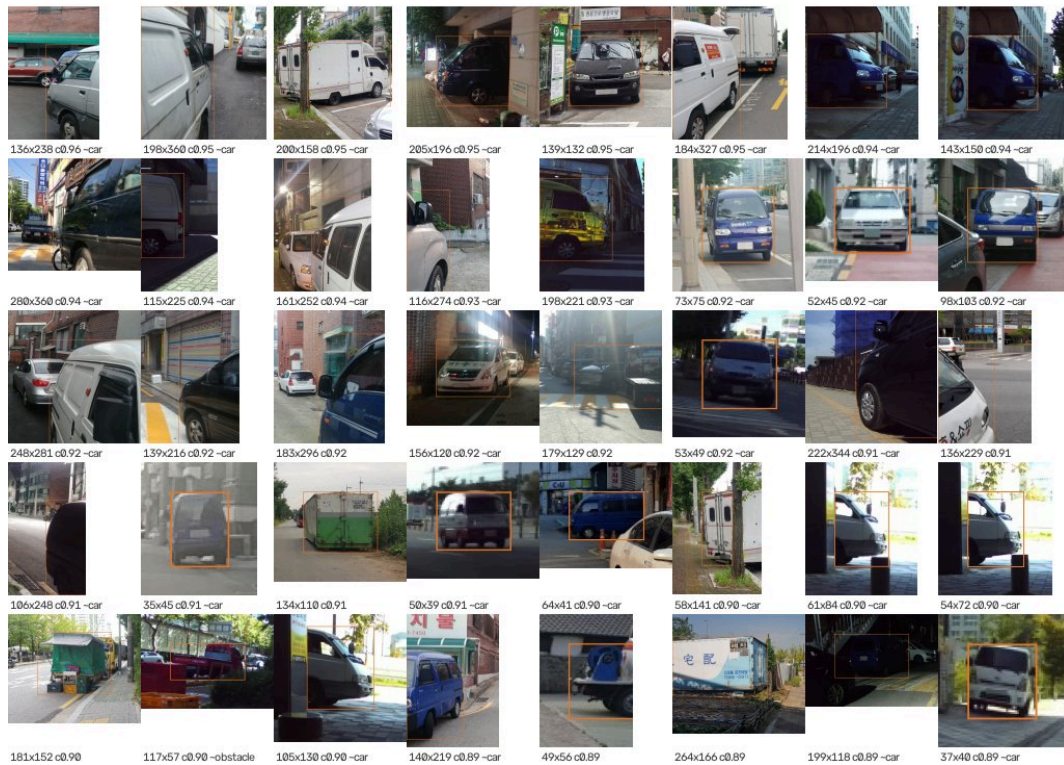
other_vehicle: 놓친 객체 (2,139개 중 무작위 40)

other_vehicle: missed, random 40 of 2139



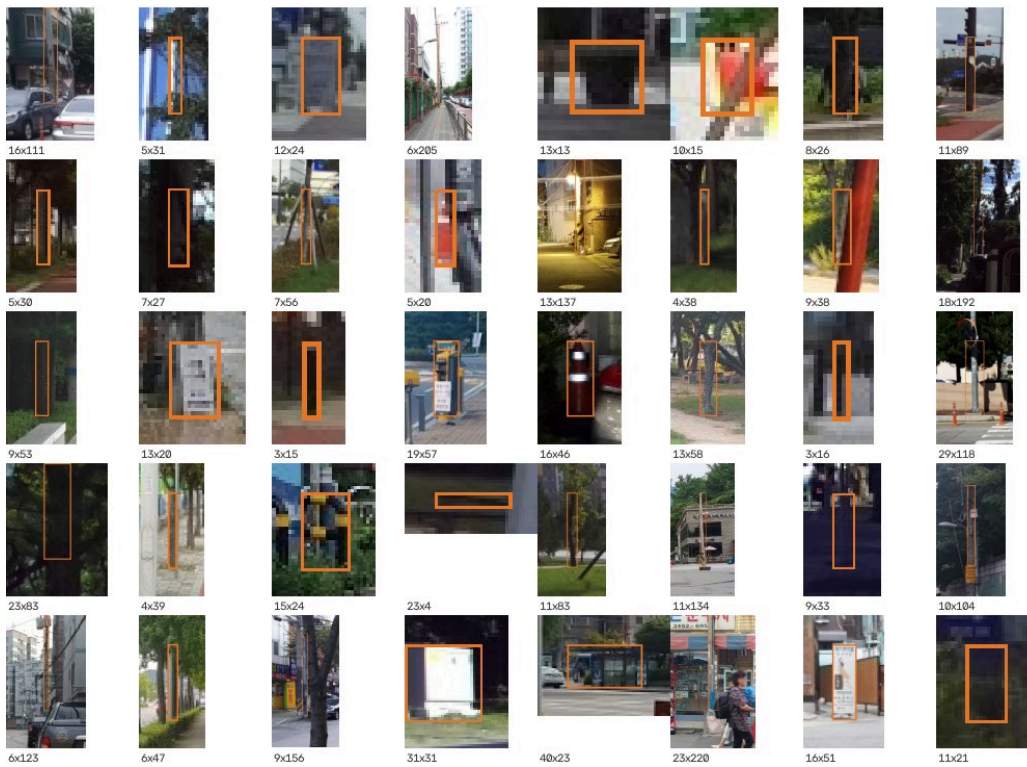
other_vehicle: 오검출 (confidence 상위 40 / 1,385개)

other_vehicle: false positives, top 40 conf of 1385



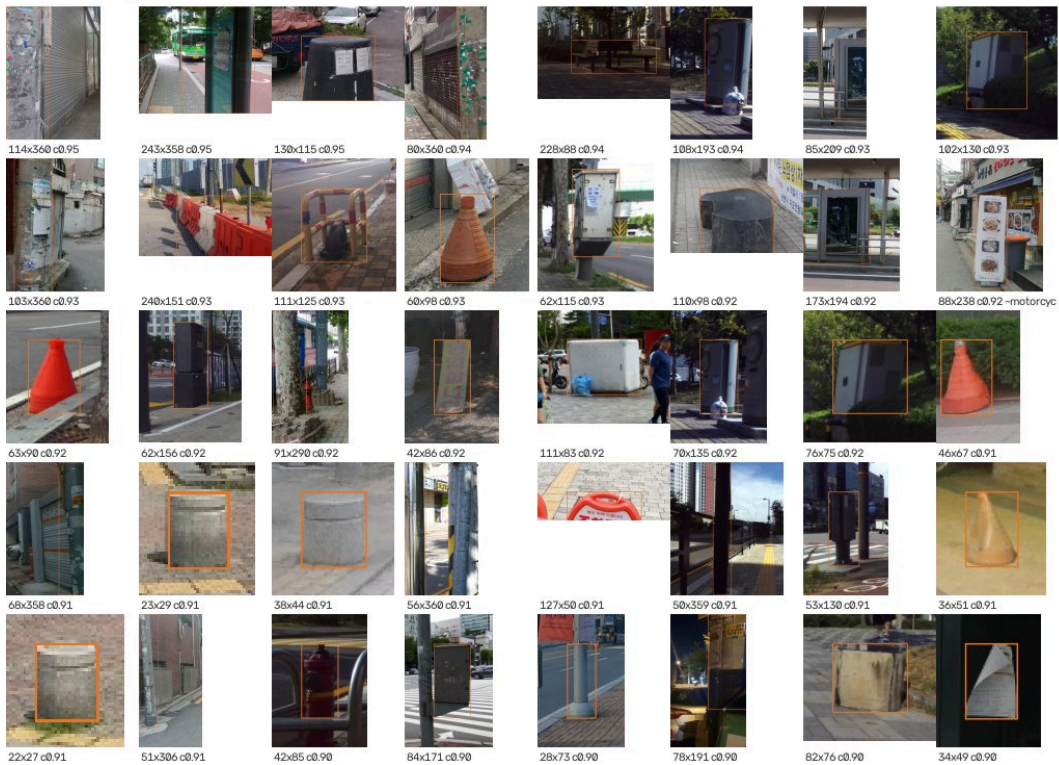
obstacle: 놓친 객체 (14,851개 중 무작위 40)

obstacle: missed, random 40 of 14851



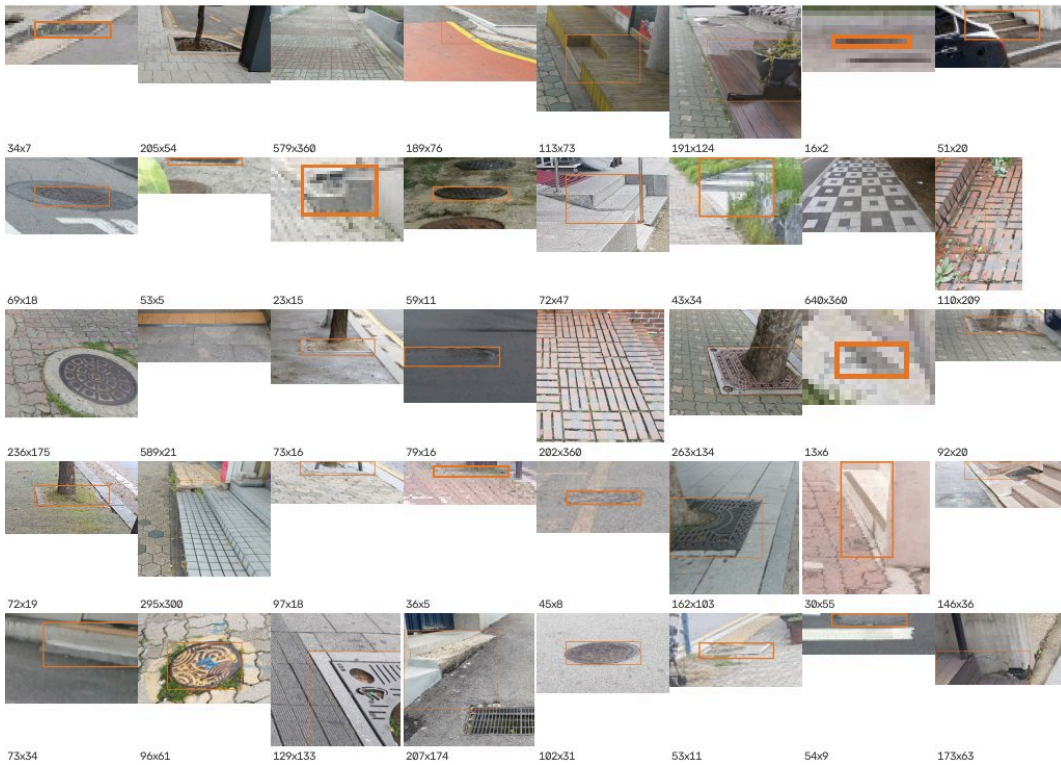
obstacle: 오검출 (confidence 상위 40 / 13,887개)

obstacle: false positives, top 40 conf of 13887



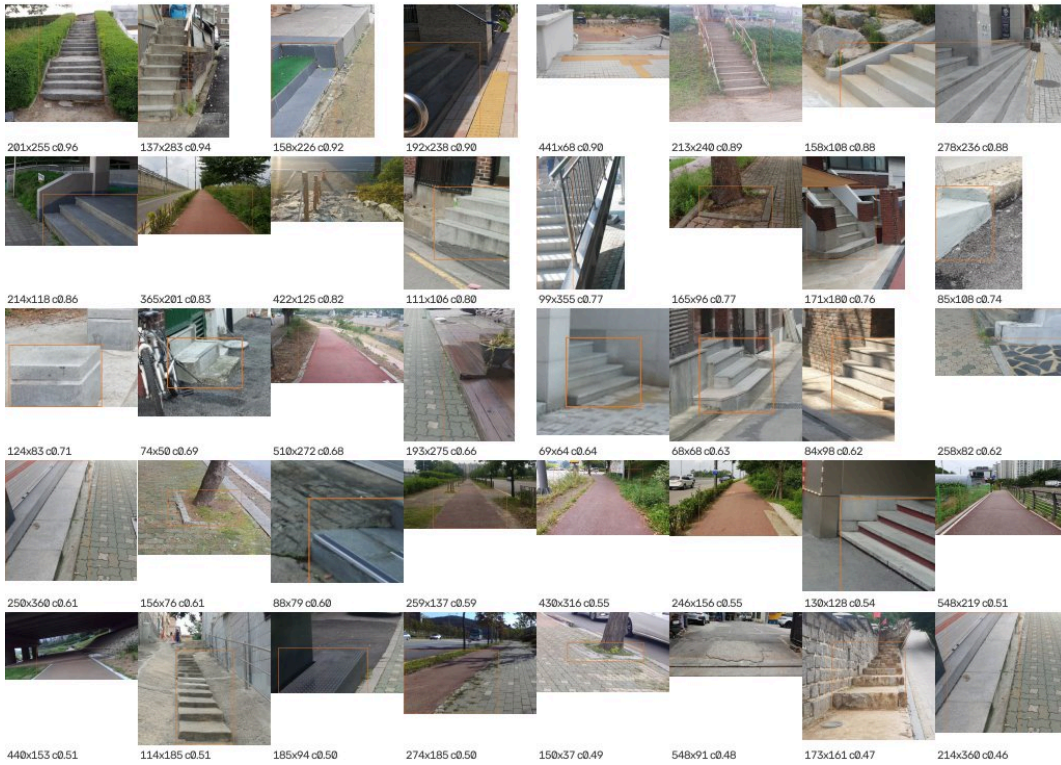
stairs: 놓친 객체 (50개 중 무작위 40)

stairs: missed, random 40 of 50



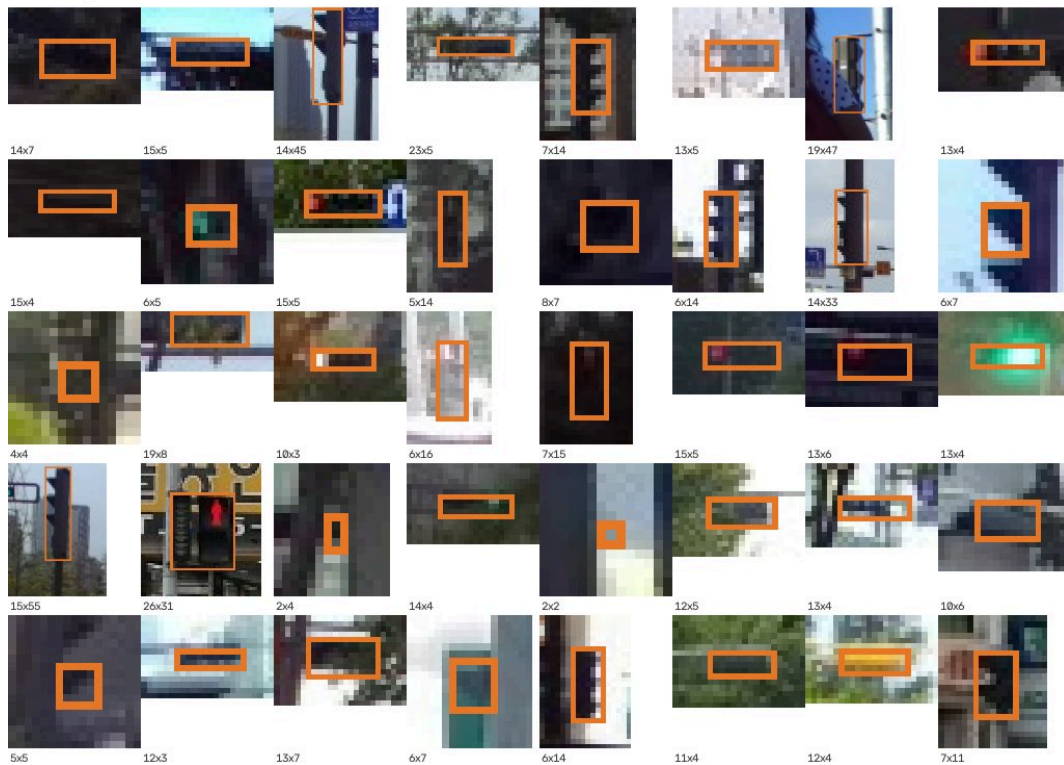
stairs: 오검출 (confidence 상위 40 / 74개)

stairs: false positives, top 40 conf of 74



traffic_light: 놓친 객체 (963개 중 무작위 40)

traffic_light: missed, random 40 of 963



traffic_light: 오검출 (confidence 상위 40 / 3,616개)

traffic_light: false positives, top 40 conf of 3616



테스트 이미지 중 클래스가 하나라도 있는 사진 수: person 4,054, bicycle 1,031, scooter 33, motorcycle 1,025, car 8,307, bus 1,138, other_vehicle 3,286, obstacle 12,123, stairs 72, traffic_light 1,680

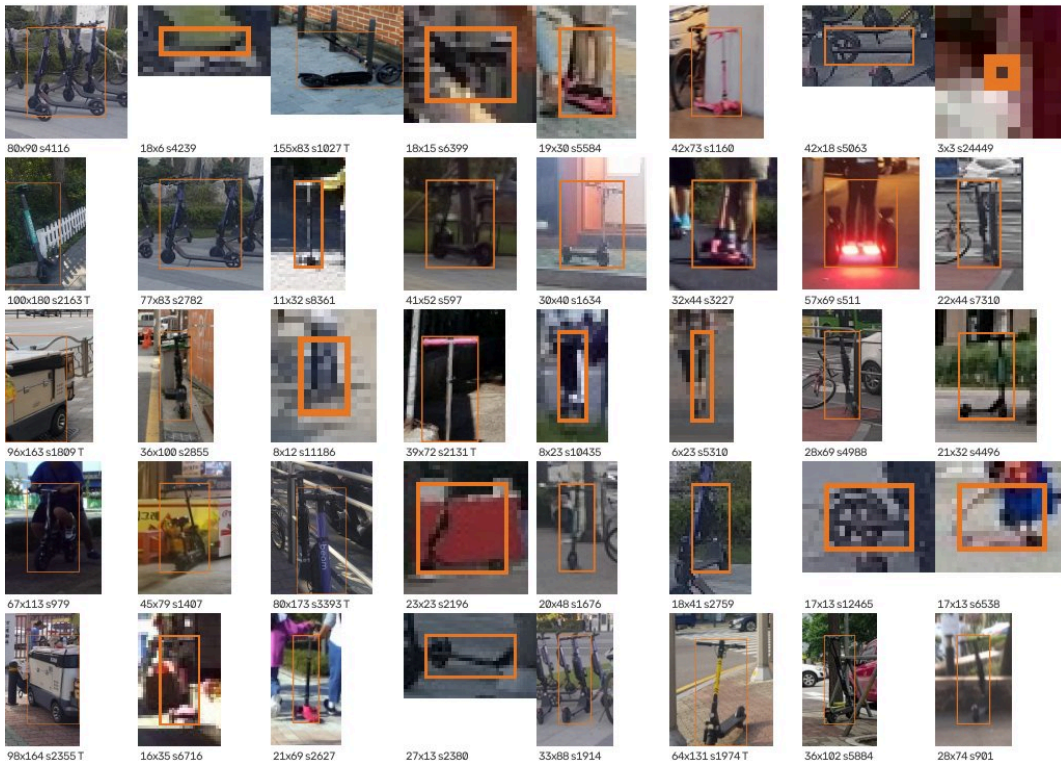
부록 E. 데이터 예시 이미지

실제 학습 이미지(640px)에서 무작위로 뽑은 객체. 파란 박스가 라벨, 아래 숫자는 박스 크기(px) / 선명도 / T= 화면 가장자리에서 잘림.

클래스별 실제 모습

scooter

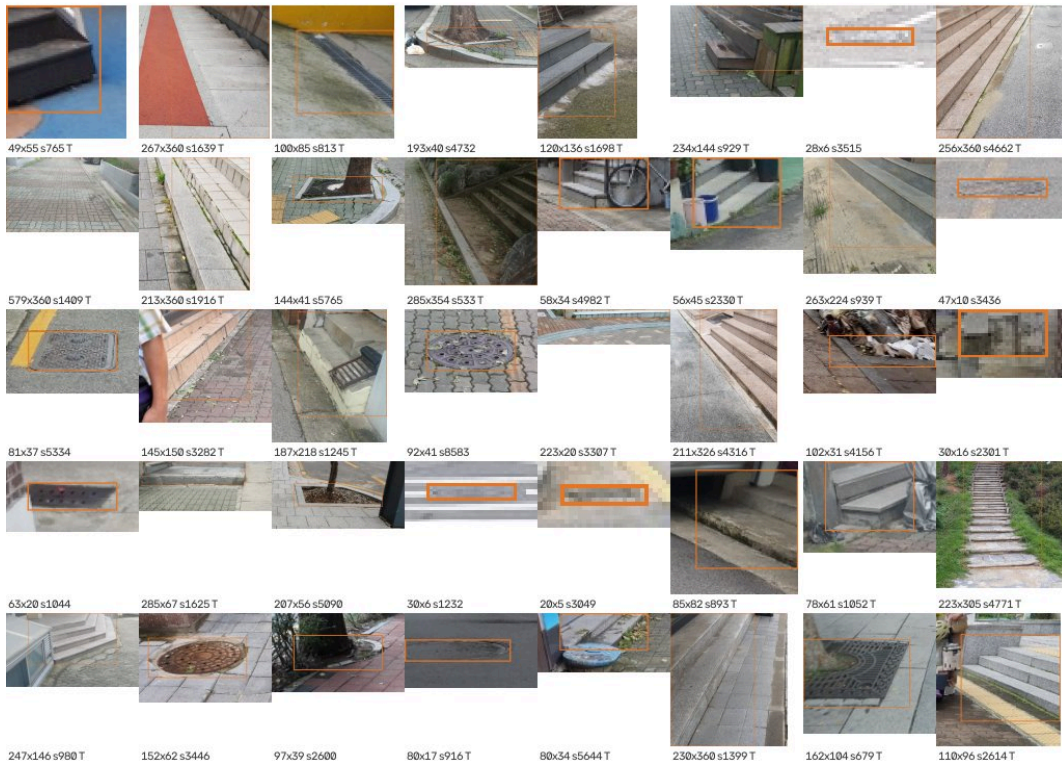
scooter: random 40



scooter 실제 객체 예시

stairs

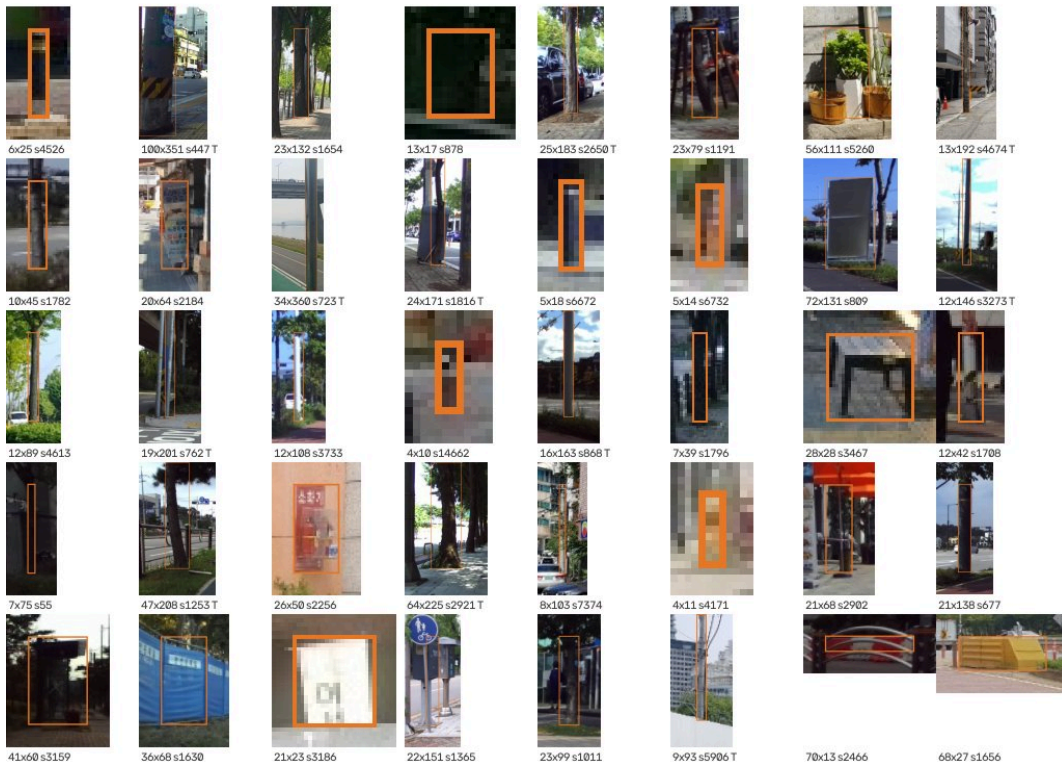
stairs: random 40



stairs 실제 객체 예시

obstacle

obstacle: random 40



obstacle 실제 객체 예시

other_vehicle

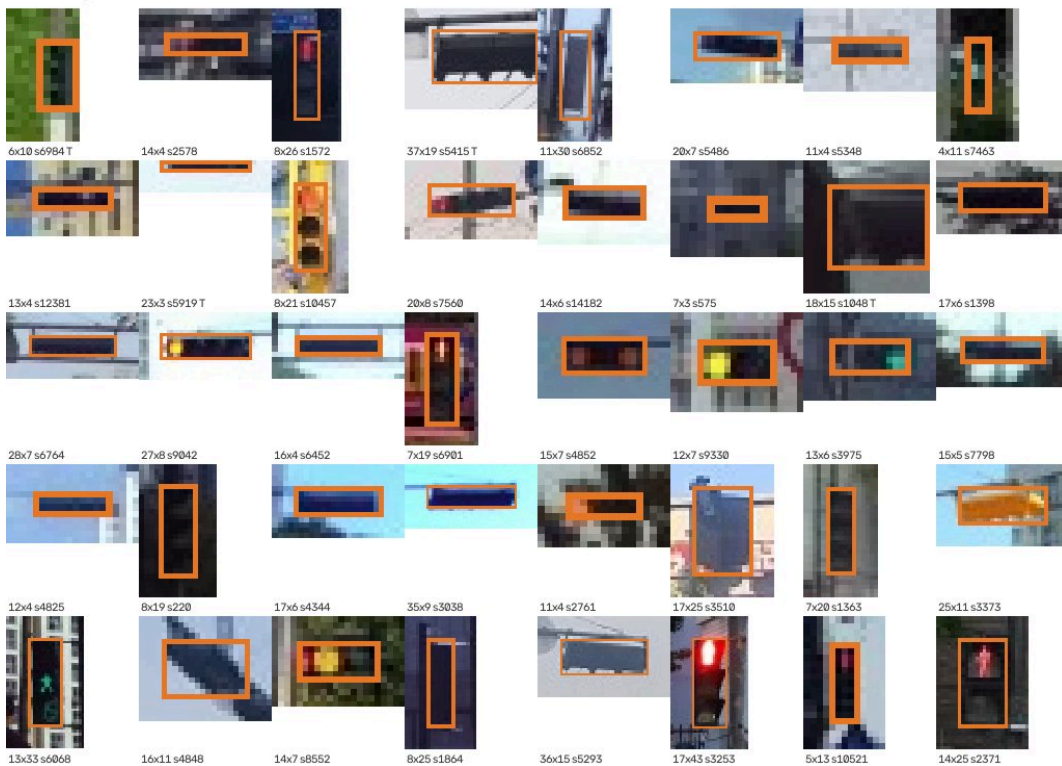
other_vehicle: random 40



other_vehicle 실제 객체 예시

traffic_light

traffic_light: random 40



traffic_light 실제 객체 예시

문제 유형별

아주 작은 객체

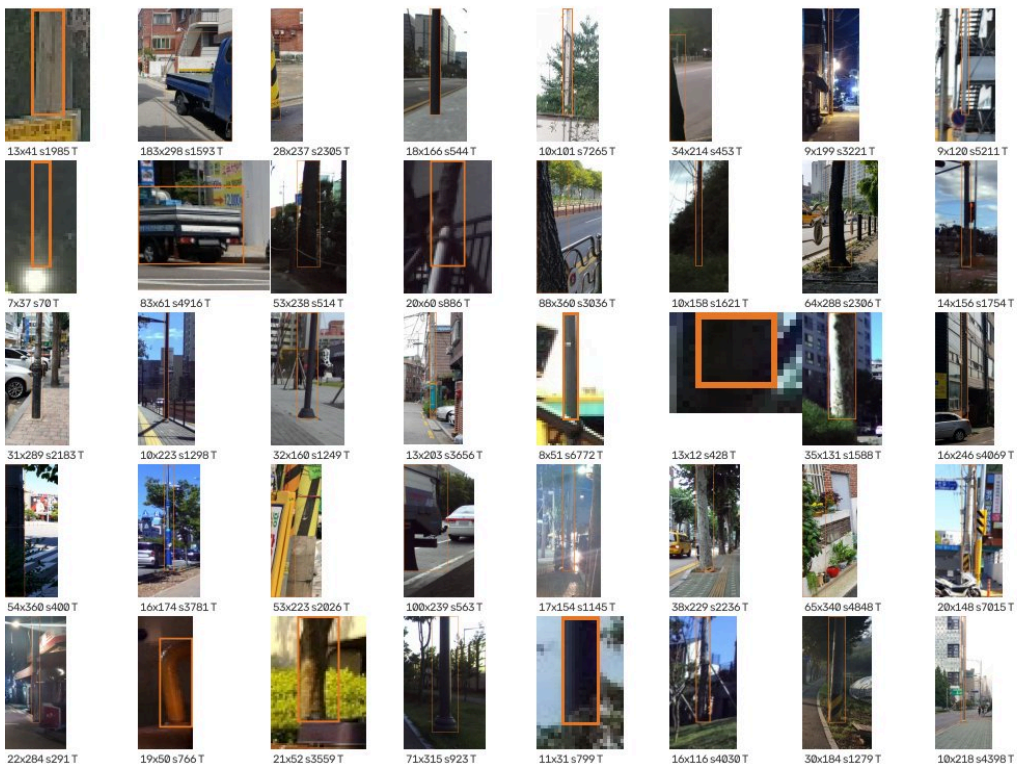
tiny: random 40 of 170699



아주 작은 객체

잘린 객체

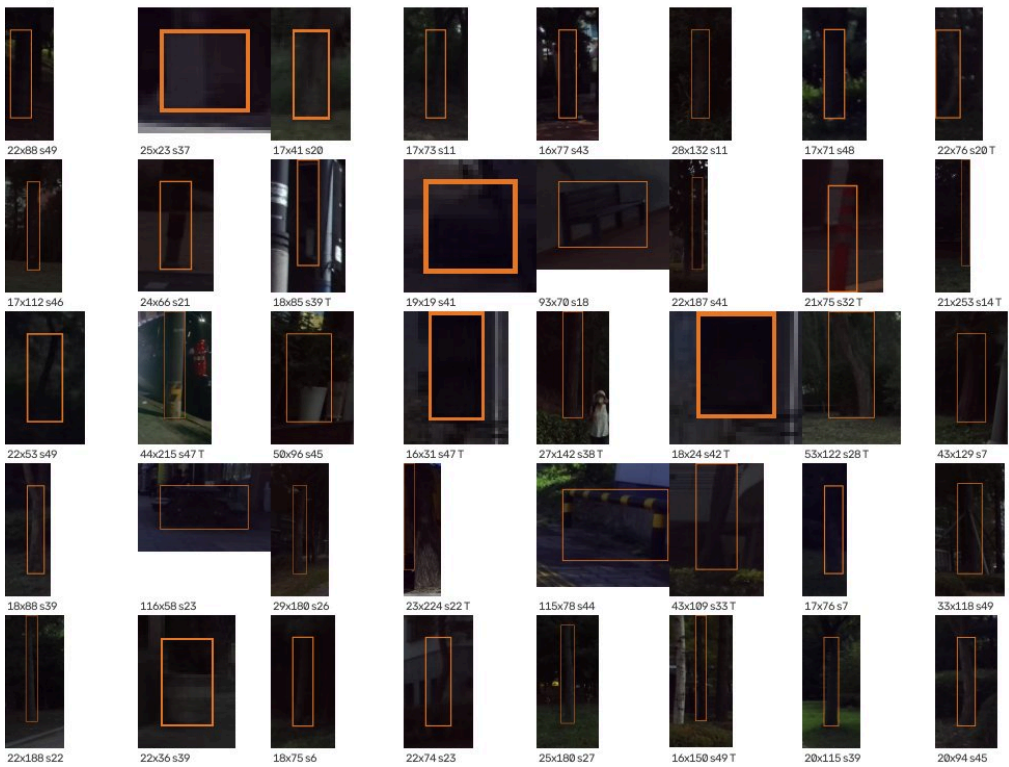
truncated: random 40 of 97428



잘린 객체

흐린 객체

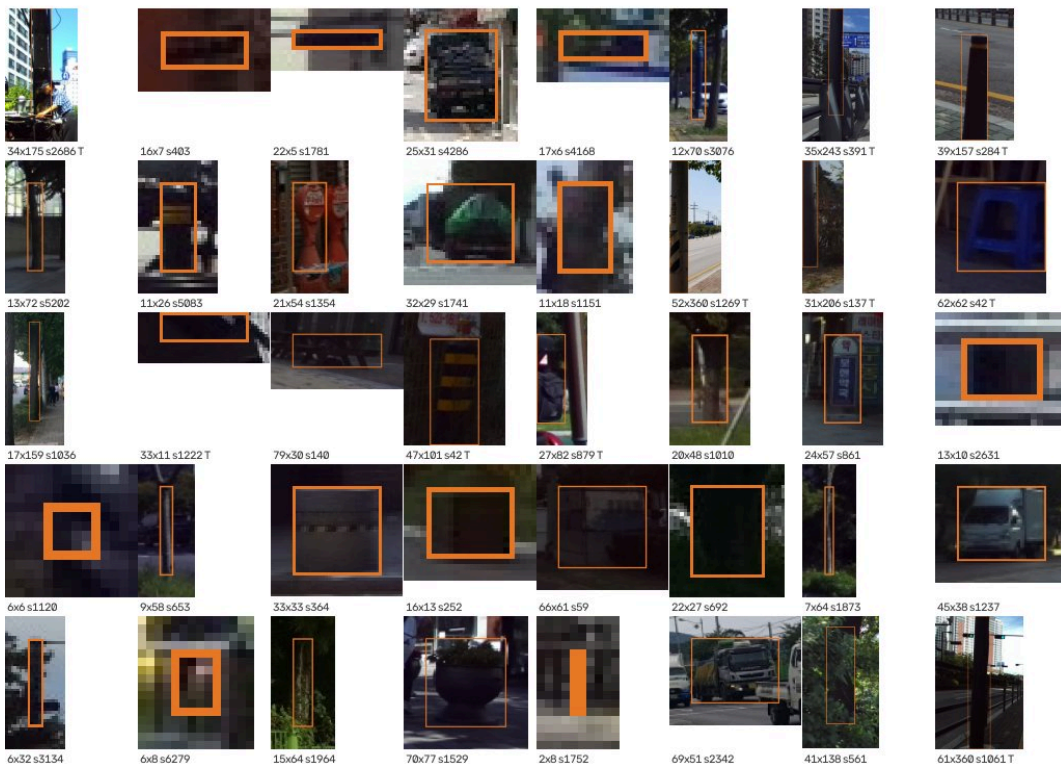
blurry: random 40 of 926



흐린 객체

어두운 객체

dark: random 40 of 76158



어두운 객체

stairs 원본 라벨 (빨간 선, AI Hub 원본 이미지)



stairs 원본 라벨

부록 F. 결과물 목록 (insight_final.zip, Drive)

파일	내용
HANDOVER.md	이 문서 본문 (인수인계 문서)
E0/best.pt , A5/best.pt	최종 모델 가중치 (각 20MB, 23MB). A5는 실행 시 C-RADIOv3 가중치를 자동으로 받음
final/final_summary.csv	최종 성능표 (클래스별 AP 포함)
final/done_E0.json , done_A5.json	모델별 전체 지표
final/results_E0.csv , results_A5.csv	epoch별 학습 기록
final/speed.md , speed.csv	추론 속도 측정
final/errors_E0/ , errors_A5/	오류 분석 보고서와 클래스별 놓친 것·오검출 예시 이미지
final/learning_curve.png , confusion_*.png	학습 곡선, 혼동 행렬
Google Drive sideguide/runs_final10/	위 전부 + 학습 로그 + 객체별 오류 원본(gt.csv, fp.csv)
Google Drive sideguide/runs/	1차 pilot 17개 모델과 로그
Google Drive sideguide/analysis/object_s.csv.gz	객체 58만 개 측정값